

Master's Programme in Mechanical Engineering

Translating Natural-Language Maritime Regulations into Machine-Readable Rules Using Large Language Models

Saad Ahmed Rana

© 2026

This work is licensed under a [Creative Commons](#)
“Attribution-NonCommercial-ShareAlike 4.0 International” license.



Author Saad Ahmed Rana

Title Translating Natural-Language Maritime Regulations into Machine-Readable Rules Using Large Language Models

Degree programme Mechanical Engineering

Major Mechatronics

Supervisor Asst. Prof. Raine Viitala

Advisors D. Sc. (Tech.) Riku Ala-Laurinaho, M. Sc. (Tech.) Haitham Al-Shami

Collaborative partner Rauma Marine Constructions

Date 28 June 2026

Number of pages 60+20

Language English

Abstract

Maritime regulations are usually written in natural language, but automated compliance verification requires machine-executable rules that software can apply to structured design data. One approach to such verification is to use Semantic Web technologies, where Resource Description Framework (RDF) graphs represent engineering data and Shapes Constraint Language (SHACL) constraints define executable validation rules. This thesis studies whether large language models can help convert selected Finnish Transport and Communications Agency (TRAFICOM) ice-class requirements into executable SHACL constraints. The thesis develops the Natural-Language-to-Logic Pipeline (NLTL), which generates SHACL constraints serialized in Turtle from structured regulation cases, verifies their Turtle syntax with RDFLib, executes them against an RDF ship design graph serialized in Turtle using pySHACL, and uses validator feedback to repair rejected candidates. The evaluation used 90 regulatory cases grouped as static, static calculation, complex readiness, and dynamic readiness cases. Each produced SHACL artifact was manually checked for semantic correctness, and this manual assessment was used as the ground truth for the reported accuracy metrics. Results are reported as mean values across three runs, with sample standard deviations reported in percentage points (pp). The developed pipeline achieved a $59.3\% \pm 1.7$ pp output rate, $66.9\% \pm 3.1$ pp accuracy among produced outputs, and $39.6\% \pm 1.3$ pp overall accuracy. In the ablation study, removing both validator-guided repair and ship-graph context reduced overall accuracy to $8.5\% \pm 2.6$ pp, showing that grounding in the target ship graph was essential. Removing only the validator increased output coverage but reduced selectivity. The results show that LLM-assisted SHACL generation can support regulatory rule authoring when grounded in the target data model and verified through execution. However, the generated rules should be treated as reviewable SHACL drafts for expert rule authoring, not as automatically approved compliance decisions, because an executable rule can still omit required applicability conditions, use incomplete ship-graph information, or encode a formula incorrectly.

Keywords natural language to logic translation, regulatory compliance, maritime regulations, large language models, SHACL, automated validation

Preface and acknowledgements

“That the powerful play goes on, and you may contribute a verse.”

— Walt Whitman

During the preparation of this thesis, artificial intelligence tools were used for language editing, grammar correction, LaTeX troubleshooting, and improving the clarity and structure of the text. In addition, large language models were used as the experimental technology studied in this thesis, as described in the methodology.

Outside this described experimental use, artificial intelligence tools were not used to produce scientific evidence, independently interpret results, make manual correctness judgments, or generate figures or images. All data interpretation, scientific reasoning, experimental design, evaluation decisions, and final thesis content were reviewed and approved by the author, who takes full responsibility for the work.

I would like to express my sincere gratitude to my supervisor, Assistant Professor Raine Viitala, for his support, valuable insights, and creative ideas throughout this thesis. I am especially grateful for the way he helped me find a path forward whenever this project presented a new challenge. I would also like to thank Riku Ala-Laurinaho and Haitham Al-Shami for their guidance, patience, and practical support throughout this work.

I would also like to thank my family and friends for always being there when I needed them the most. I am especially grateful to my parents, without whom I could never have made it this far, and to my brothers for their constant support throughout my journey. I would also like to thank Srijit Bashyal, Hugo Tamm, and Lucas Foley, who have seen the ups and downs of this project and my life in general, and have always remained a constant source of support.

Writing this thesis has been a meaningful journey. Along the way, I have learned a great deal, faced many challenges, and made connections and friendships that I will carry with me for the rest of my life. Thank you to everyone who supported me when I needed it most. I hope to take this research further in the future and contribute to its development beyond the scope of this thesis.

Espoo, 28 June 2026

Saad Ahmed Rana

Contents

Abstract	3
Preface and acknowledgements	4
Contents	5
Symbols and abbreviations	7
1 Introduction	8
1.1 Research problem	10
1.2 Aim of the research	11
1.3 Scope and limitations	12
1.4 Methods	13
1.5 Structure	14
2 Background and state of the art	15
2.1 Need for formal validation rules	15
2.2 Technologies for graph-based validation: RDF and SHACL	17
2.3 Rule authoring approaches: manual, template-based, and LLM-assisted	19
2.4 State of the art: grounding and verification-guided generation	20
2.5 Research gap and positioning of this thesis	22
3 Materials and methods	25
3.1 NLTL pipeline	25
3.2 Experimental material and case representation	28
3.3 Evaluation procedure	30
4 Case study and evaluation	35
4.1 Case-study setup and experimental configurations	36
4.2 Full NLTL pipeline repeated-run results	37
4.3 Ablation studies	38
4.4 Failure-mode analysis	39
5 Discussion	43
5.1 Analysis of the results	43
5.2 Comparison to earlier research	45
5.3 Uncertainty and error analysis	46
5.4 Scientific impact	48
5.5 Practical impact	49
5.6 Future research and open research questions	50
6 Conclusions	52
References	54

A Few-shot SHACL examples used for prompting	61
B Natural-language regulatory inputs used in the benchmark	64
C Generated SHACL outputs for selected benchmark cases	70
D Prompt templates used in the experiments	74

Symbols and abbreviations

Symbols

N_{total}	Total number of benchmark cases
N_{produced}	Number of cases for which an output was produced
N_{correct}	Number of produced outputs judged correct

Abbreviations

NLTL	Natural-Language-to-Logic Pipeline
AML	Automation Markup Language
FMI	Functional Mock-up Interface
SSP	System Structure and Parameterization
ReqTLo	Requirements Temporal Logic ontology
BIM	Building Information Modeling
DSR	Design Science Research
GPT	Generative Pre-trained Transformer
JSON	JavaScript Object Notation
JSON-LD	JavaScript Object Notation for Linking Data
LLM	Large Language Model
MCR	Maximum Continuous Rating
NLP	Natural Language Processing
ODRL	Open Digital Rights Language
OEM	Original Equipment Manufacturer
OWL	Web Ontology Language
RDF	Resource Description Framework
RDFLib	Python library for working with RDF data
RDFS	RDF Schema
RQ	Research Question
SHACL	Shapes Constraint Language
SPARQL	SPARQL Protocol and RDF Query Language
SysML	Systems Modeling Language
TRAFICOM	Finnish Transport and Communications Agency
Turtle	Terse RDF Triple Language
W3C	World Wide Web Consortium
pySHACL	Python validator for SHACL constraints
pp	percentage points
RAG	Retrieval-Augmented Generation

1 Introduction

Modern mechatronic and mechanical systems are increasingly developed through distributed engineering processes. In these processes, several organizations and engineering disciplines contribute to design models, analyses, documentation, and verification evidence. Design assumptions and compliance evidence must therefore be communicated across organizational and disciplinary boundaries. Recent work on Digital Twin Web for mechanical-system design argues that automated and collaborative design requires moving beyond manual document exchange toward standardised digital representations of systems, components, and analyses [1].

The transition toward distributed engineering motivates workflows in which design information is represented explicitly and can be verified systematically. In such workflows, requirements, design data, and verification results can be linked in a traceable manner. Shared standards and data formats are important because they support the exchange of models, system structures, and engineering data between tools and organizations. For example, the Functional Mock-up Interface (FMI) supports tool-independent exchange of simulation models, including co-simulation components that can be coupled with other tools, while the System Structure and Parameterization (SSP) standard supports exchange of system structures and parameterization information [2, 3]. In the maritime domain, ISO 19848 provides a standard structure for naming and describing shipboard machinery and equipment data [4]. These standards illustrate why shared identifiers, structured data, and machine-readable validation artifacts are important for collaborative engineering.

Recent maritime-engineering research has begun to turn these standards into automation-supporting design workflows. Sivic et al. present DDTLab, a web-based platform that transforms SysML v2 conceptual models into SSP system structures for maritime co-simulation [5]. Muhammad evaluated a multimodal retrieval-augmented generation system for assisting engineers with classification standards by retrieving text, equations, tables, and images from classification-society documents and generating grounded natural-language answers with source references [6]. AnchorMap addresses a complementary vocabulary-alignment problem by mapping heterogeneous OEM, simulation, and regulatory variable labels to canonical identifiers [7]. Together, these works illustrate a broader movement toward structured, traceable, and AI-assisted engineering workflows. The present thesis follows this direction but focuses on a different bottleneck: translating natural-language maritime regulations into executable SHACL validation rules.

A similar challenge appears in regulatory compliance and classification. In maritime engineering, ship designs are assessed against statutory regulations, flag-state requirements, and classification-society rules. These rule sets define technical conditions for vessel design, machinery, safety, operation, and ice-class performance. They are usually written in natural language, often combining normative expressions such as “shall” or “must” with engineering quantities, exceptions, applicability conditions, formulae, and references to other parts of the regulation or rule set. In practice, engineers must interpret these clauses, identify the relevant design variables, and compare them with calculations, design data, simulation outputs, or Original

Equipment Manufacturer (OEM) documentation. This manual process is time-consuming, difficult to scale across large rule sets, and vulnerable to ambiguity, inconsistent interpretation, and human error. This thesis studies this general natural-language-to-logic problem through a maritime case study based on selected ice-class requirements from the Finnish Transport and Communications Agency (TRAFICOM) regulation [8].

A promising approach to scalable compliance checking is to represent engineering data in machine-readable form and validate it using explicit executable constraints. In this thesis, an executable constraint means a formal validation rule that software can run against structured engineering data, for example to check whether a required value exists or whether a value satisfies a regulatory threshold. The Shapes Constraint Language (SHACL) is used as the target constraint language because it can express validation rules over Resource Description Framework (RDF) graphs and can be executed using existing tools such as the pySHACL validator. This makes it possible to connect regulatory requirements to structured ship design data and to produce validation evidence that can be inspected after execution.

However, SHACL-based validation in ship design depends on more than the constraint language alone. It also requires a shared vocabulary for ship components and engineering variables, together with clear RDF paths for reading their values from the graph. A generated rule is only useful if it refers to identifiers and paths that actually exist in the target data model. Previous work provides two parts of this foundation. The Requirements Temporal Logic ontology (ReqTLo) represents dynamic requirements and links them to simulation artifacts, and it also uses SHACL rules to validate the consistency of requirement specifications [9]. AnchorMap addresses the related variable-standardisation problem by mapping heterogeneous OEM, simulation, and regulatory variable labels to canonical identifiers in a linked-data schema using embedding retrieval, large-language-model-based reasoning, confidence routing, and human review [7]. These works motivate the setting of the present thesis: once requirements and engineering variables can be represented with shared identifiers, a remaining bottleneck is the authoring of executable validation rules from natural-language regulation.

Building on this foundation, a key remaining bottleneck is the translation of natural-language maritime regulations into executable validation logic. Manually authoring SHACL constraints for regulatory clauses is costly and difficult to maintain as requirements, data models, and design variants evolve. This thesis addresses that bottleneck by developing and evaluating an LLM-assisted pipeline, the Natural-Language-to-Logic Pipeline (NLTL), that translates selected TRAFICOM ice-class requirements into executable SHACL constraints and validates them against an RDF ship design graph serialized in Turtle using a generation–execution–repair workflow. Figure 1 summarizes the high-level context of this work, from engineering and regulatory inputs to machine-readable rule generation and design validation.

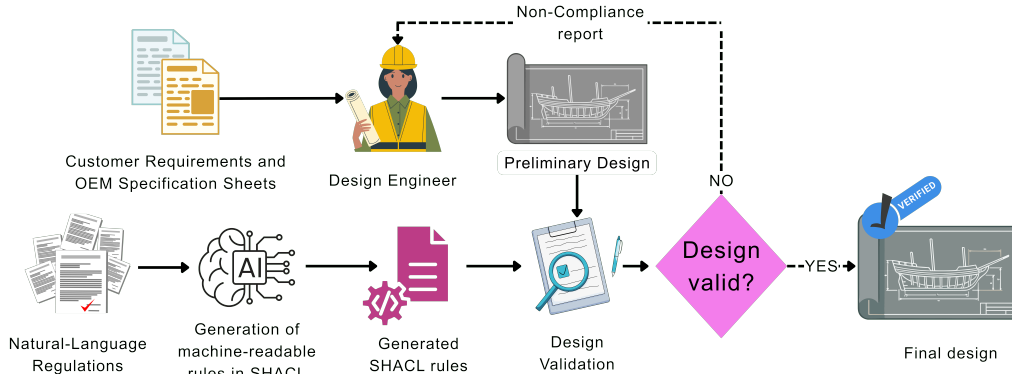


Figure 1: Conceptual overview of the proposed compliance-checking workflow. Customer requirements and OEM specifications guide the preliminary ship design. In this figure, NLTL corresponds to the rule-generation and validation part of the workflow: it translates selected natural-language maritime regulations into machine-readable SHACL rules and executes them against the RDF ship design graph serialized in Turtle. The resulting validation report can then be inspected by the design engineer and used to support design revision when a selected requirement is not satisfied.

1.1 Research problem

Most regulatory requirements that govern engineering design are written in natural language, yet automated compliance checking requires those requirements to be expressed as precise, machine-executable rules. This mismatch creates a costly manual step in which experts interpret clauses, identify relevant engineering variables, resolve applicability conditions, and encode the intended meaning into formal constraints that can be executed against structured design data.

In SHACL-based validation workflows, constraints are usually created by experts, often with specialized semantic-web tools. Once a SHACL rule has been authored, the execution workflow is similar whether the rule was written manually or generated by an LLM. The bottleneck addressed in this thesis is therefore the preceding translation step: moving from a natural-language regulatory clause to an executable SHACL rule that matches the target RDF data model. This translation is difficult because a syntactically valid SHACL shape may still be incorrect if it targets the wrong node, uses a nonexistent RDF path, omits an applicability condition, or refers to variable names that are not present in the ship graph.

Previous research has addressed parts of this translation problem through manually authored checking rules in automated compliance-checking systems [10, 11], NLP- and template-based rule extraction [12, 13], ontology-derived SHACL generation [14], and more recent LLM-assisted generation of formal artifacts [15, 16]. However, these approaches either require substantial manual rule engineering, are limited to predefined templates or existing schema constraints, or do not fully evaluate whether the generated constraints execute correctly against a specific RDF engineering data model. This leaves an open need for a workflow that combines natural-language regulatory input, target-graph grounding, deterministic execution checks, and semantic

review of the generated rule artifact.

Therefore, the research problem addressed in this thesis is how to support the translation of selected natural-language maritime regulatory requirements into executable SHACL constraints using an LLM-assisted generation, validation, and repair workflow. The focus is not on all possible ways of formalizing regulations, but specifically on whether structured regulation cases, target ship-graph context, deterministic SHACL execution, and validator-guided feedback can produce useful reviewable SHACL drafts for maritime design validation.

To structure this problem, the thesis is guided by the following research questions (RQs):

- **RQ1.** How can LLMs be used to generate SHACL constraints that are executable on RDF ship graphs from selected natural-language maritime regulations?
- **RQ2.** How can the LLM-generated SHACL artifacts be checked for syntactic validity, executability on RDF ship graphs, and alignment with the target ship data model?
- **RQ3.** How do ship-graph grounding and validator-guided repair affect the quality, correctness, and limitations of the generated SHACL constraints?

1.2 Aim of the research

The aim of this thesis is to develop and evaluate an LLM-assisted workflow for translating selected natural-language maritime regulatory requirements into executable SHACL constraints for an RDF ship design graph. The thesis does not only test whether an LLM can produce plausible SHACL text. Instead, it studies whether generated rules can be extracted, parsed, executed against the target graph, repaired using feedback, and manually assessed for semantic correctness.

The work is designed to remain relevant even as LLM capabilities improve. The central problem is not only the language model itself, but the engineering workflow around it: how regulatory text is represented as cases, how target-graph context is provided, how generated SHACL is checked with deterministic tools, how feedback is used for repair, and how correctness is judged against the intended regulatory meaning. Better future models may improve individual generation quality, but they will still need grounding, execution checks, traceability, and expert review when used for regulatory rule authoring.

More specifically, the thesis was conducted through four practical steps. First, selected TRAFICOM ice-class requirements were represented as structured regulatory cases. These cases provide the input to the LLM-based generation process and support traceability between the source requirement, generated SHACL artifact, execution result, and manual evaluation. This step supports RQ1 by defining how selected natural-language regulations are used as inputs for LLM-based SHACL generation.

Second, the NLTL pipeline was implemented to generate SHACL constraints serialized in Turtle from structured case files, few-shot examples, and ship-graph context. The generated artifacts are extracted from the LLM output, checked for

Turtle syntax validity using RDFLib, and executed against the ship graph using pySHACL. This step supports RQ2 by checking whether generated SHACL artifacts are syntactically valid, executable on RDF ship graphs, and aligned with the target ship data model.

Third, a validator-guided repair loop was added to assess and refine generated candidates. The validator model receives the regulation case, generated SHACL, syntax result, SHACL execution result, and ship-graph context, and then decides whether the candidate should be accepted or repaired. If the candidate is not accepted, the validator’s issues and suggested fix are passed back to the generator in the next iteration. This step supports RQ3 by allowing the effect of validator-guided repair to be evaluated.

Finally, the implemented pipeline was evaluated on a benchmark set of 90 regulatory cases. The evaluation reports output rate, accuracy among produced outputs, overall accuracy, repeated-run mean and sample standard deviation, ablation comparisons, and manually assigned failure modes. The ablation comparison is used to separate the effect of ship-graph grounding from the effect of validator-guided repair.

1.3 Scope and limitations

The scope of this thesis is limited to selected maritime regulatory requirements from the Finnish Transport and Communications Agency (TRAFICOM). The primary source is the TRAFICOM regulation *Ice Class Regulations and the Application Thereof* (TRAFICOM/68863/03.04.01.00/2021), issued on 1 July 2021 and in force from 5 July 2021 [8]. Regulations from other authorities, classification societies, and other maritime domains are outside the scope of this work.

The thesis does not attempt to formalize the full TRAFICOM ice-class regulation or to provide a complete regulatory compliance system. Instead, the evaluation uses selected requirements that can be represented as individual validation cases and tested against an RDF ship design graph. The results are therefore limited to the selected benchmark cases, the available ship graph, and the vocabulary used in that graph.

The selected requirements mainly cover conditions that can be checked against available ship design data. These include direct value checks, threshold checks, enumeration checks, simple formula-related checks, and readiness checks over variables represented in the ship graph. Full simulation-based verification is outside the scope of this thesis. Requirements that depend on time-domain behaviour, physical simulation, fatigue analysis, vibration analysis, or digital-twin execution are not evaluated as complete dynamic compliance checks. However, selected dynamic or analysis-dependent requirements are included when they can be treated as readiness checks, meaning that the generated SHACL verifies whether required output variables or analysis slots are present in the RDF graph.

With respect to the form in which regulatory requirements are expressed, Table 1 summarizes the requirement types considered in this thesis and how they are treated. The pipeline assumes that the relevant ship design information is already represented in a predefined RDF graph serialized in Turtle using a controlled vocabulary for components, variables, and values. The work does not automatically extend the ship

Table 1: Scope of regulatory requirement types considered in this thesis.

Regulation type	Description	Treatment
Natural-language constraints	Textual obligations such as “shall have” or “must not exceed”.	Included
Arithmetic and threshold constraints	Numeric thresholds, ranges, inequalities, or simple value comparisons.	Included
Formula-based constraints	Equations, lookup values, or computed quantities requiring variables, assumptions, or precomputed values.	Included when simple or stored
Complex readiness requirements	Checks whether required variables, outputs, or structural information exist in the ship graph.	Readiness checks
Dynamic readiness requirements	Time-domain simulation, fatigue, vibration, propulsion response, or other dynamic analyses.	Readiness only
Visual or diagram-based requirements	Requirements expressed mainly through figures, diagrams, schematics, or visual interpretation.	Excluded

graph, discover missing domain concepts, or infer new engineering variables during execution. If a requirement refers to information that is not represented in the graph, the generated SHACL cannot infer or retrieve a closest matching variable during execution. It can only check variables and RDF paths that are actually present, or expose the missing information as a limitation of the available ship graph.

The generated SHACL constraints are evaluated as reviewable rule drafts: executable artifacts that can be read, executed, traced back to the source case, and manually checked by experts. Within the scope of this thesis, they are not treated as final evidence of regulatory compliance, classification-society assessment, or formal approval. A generated rule may execute successfully while still encoding only part of the intended requirement, depending on the available ship-graph data, the interpretation of when the requirement applies, and the correctness of any formula or threshold logic. The applicability of the approach to other regulatory documents, other ship graphs, or substantially different data models is not assessed in this thesis.

1.4 Methods

This thesis uses a design-oriented research approach supported by a literature review and a maritime case-study evaluation. The literature review identifies the need for machine-readable regulatory rules, reviews RDF and SHACL as graph-based validation technologies, and positions LLM-assisted rule generation in relation to earlier work on automated compliance checking, semantic-web validation, grounding, and verification-guided generation.

The empirical part of the thesis follows a Design Science Research (DSR) logic, where a designed artifact is built and evaluated. The artifact developed in this work is NLTL, an LLM-assisted pipeline for generating executable SHACL constraints from selected natural-language maritime regulatory requirements. The pipeline is evaluated in a controlled case study using selected TRAFICOM-derived regulatory cases and an

RDF ship design graph.

The evaluation combines automated tool-based checks, repeated experimental runs, ablation comparisons, and manual semantic assessment. Generated SHACL artifacts are extracted, parsed, executed against the RDF ship graph, and then manually assessed to determine whether they represent the intended regulatory meaning. The detailed implementation and evaluation procedure are described in Section 3. The case-study results are reported in Section 4, and the implications of the results are discussed in Section 5.

1.5 Structure

The remainder of this thesis is organized as follows. Section 2 presents the background and state of the art on automated compliance checking, formal requirement representation, SHACL-based validation, and LLM-assisted translation from natural language to executable artifacts. Section 3 describes the research material and methods, including the TRAFICOM regulatory source, the structured case representation, the RDF ship design graph, the NLTL pipeline, SHACL extraction, syntax checking, pySHACL execution, validator-guided repair, and the evaluation methodology. Section 4 reports the case-study results, including the full NLTL pipeline repeated-run results, ablation comparison, and failure-mode analysis. Section 5 discusses the findings, compares the results with earlier research, examines uncertainty and error sources, and considers scientific impact, practical implications, and future improvements. Finally, Section 6 summarizes the main contributions and provides concluding remarks.

2 Background and state of the art

This section reviews the research and technologies relevant to translating natural-language regulatory requirements into executable validation rules. The review is organized around five themes. First, it explains why human-readable regulatory prose must be transformed into formal validation rules before automated checking is possible. Second, it introduces RDF and SHACL as the graph-based data and constraint technologies used in this thesis. Third, it reviews rule-authoring approaches, ranging from manual authoring and template-based natural language processing to LLM-assisted generation. Fourth, it discusses recent work on grounding and verification-guided generation of formal artifacts. Finally, it identifies the research gap addressed by this thesis: LLM-assisted generation of executable SHACL rules for selected maritime regulatory requirements against a concrete RDF ship design graph.

2.1 Need for formal validation rules

Regulatory and technical requirements are usually written for human readers. This makes them understandable in engineering practice, but it also makes them difficult to check automatically. A software-based validation system cannot directly act on a sentence written with expressions such as “shall”, “must”, “is to be”, or “need not exceed”. The requirement must first be converted into a rule that identifies the relevant data, states the condition to be checked, and produces an interpretable validation result.

The general problem is well established in automated compliance checking. In the architecture, engineering, and construction domain, Eastman et al. describe automatic rule checking as a process in which design information is checked against formally represented rules [10]. Dimyadi and Amor similarly identify the translation of regulatory provisions into computable rules as a central bottleneck in automated building-code compliance checking [11]. This conversion is difficult because regulatory requirements often contain more than a simple value comparison. They may include applicability conditions, exceptions, cross-references, formulae, lookup tables, units, engineering variables, and assumptions that are clear to domain experts but not explicit enough for direct computational use. Bus et al. highlight several obstacles in smart building-code checking, including ambiguous or incomplete regulatory text, mismatches between regulatory vocabulary and model vocabulary, and calculations that require preprocessing of design data [17]. Noardo et al. also show that real design models may require additional interpretation and inference before common planning checks can be automated [18]. These observations are relevant to this thesis because the generated rule must not only reflect the wording of a regulation, but must also match the variables and value-access patterns present in the ship graph.

Several research streams address different parts of this problem. Natural Language Processing (NLP) has been used to extract structured information from regulatory documents. Zhang and El-Gohary propose a semantic rule-based NLP approach for extracting information from construction regulatory text for compliance checking [12]. Legal and regulatory NLP has also been studied more broadly through tools and

frameworks such as LexNLP [19] and through automated extraction of semantic legal metadata [20]. More recently, CODE-ACCORD provides an annotated corpus of building regulatory sentences intended to support machine-readable rule generation for automated compliance checking [21]. These works show that regulatory text can be processed computationally, but they also show that the extracted meaning must eventually be represented in a form that can be executed or reasoned over.

Formal rule languages and semantic representations provide another route. Legal-RuleML, for example, standardizes a markup language for representing legal normative rules [22]. Logic-based approaches have also been studied for compliance checking, including constraint logic programming and answer set programming for Building Information Modeling (BIM)-based checking [23]. These approaches demonstrate the value of executable or machine-interpretable formalisms, but they do not remove the need to connect source text, domain vocabulary, and design data.

Ontology-based requirement approaches are relevant because they provide shared terminology and explicit links between requirements, system elements, and verification evidence. ReqTLo, for example, represents requirements using a temporal-logic-oriented ontology and links them to simulation-based validation artifacts [9]. The same work also uses SHACL rules to validate the internal consistency of requirement specifications. This shows how formal requirement models can improve traceability and support automated checks at the requirement-model level.

However, a formal vocabulary or ontology does not by itself solve the rule-authoring problem. A validation system still needs concrete constraints that can be evaluated against available engineering data. For prescriptive maritime regulations, this means that the meaning of a natural-language clause must be converted into a checkable rule and connected to the identifiers used in the target data model. The TRAFICOM ice-class regulation used in this thesis contains direct thresholds, formula-based clauses, applicability conditions, machinery requirements, and analysis-dependent requirements [8]. This makes it a suitable case for studying the translation from regulatory prose into executable validation artifacts.

Requirements engineering provides an additional perspective on why this translation is difficult. Requirements engineering studies how stakeholder goals, domain assumptions, system constraints, and verification needs are captured and maintained during system development [24]. Zave and Jackson argue that requirements engineering depends on carefully distinguishing the environment, the machine, and the relationship between them [25]. This distinction is relevant to regulatory validation because a regulatory clause usually describes a required property of a ship, component, operating condition, or engineering analysis, while an executable SHACL rule checks only a particular representation of that property in an RDF data graph.

Natural-language requirements are common because they are understandable to stakeholders, but they are also prone to ambiguity, incompleteness, and unverifiable wording. Requirements-engineering standards emphasize that requirements should be necessary, unambiguous, complete, consistent, feasible, traceable, and verifiable [26]. In practice, these qualities are difficult to guarantee in long regulatory documents. Requirements-smell research addresses this problem by detecting linguistic patterns that may indicate weak, ambiguous, vague, or hard-to-verify requirements [27].

Such work supports the premise of this thesis: before a requirement can be checked automatically, its meaning and verification condition must be made explicit enough for software execution.

A related body of work studies reusable requirement and specification patterns. Dwyer et al. introduced property specification patterns to help engineers express recurring requirements for finite-state verification [28]. Konrad and Cheng extended this idea to real-time specification patterns for embedded and time-constrained systems [29]. More recent work continues this direction by providing pattern catalogs for real-time verification tools such as UPPAAL [30]. Pattern-based approaches are relevant because many regulatory clauses repeat common structures, such as thresholds, required existence of variables, bounded ranges, applicability conditions, or time-related readiness conditions.

Formalization and validation of requirements have also been studied in safety-critical domains. Cimatti et al. note that validating requirements themselves is a fundamental step in safety-critical development and that formalization often requires significant domain-expert involvement [31]. This observation also applies to the present thesis. NLTL can draft executable SHACL rules, but expert review remains necessary because syntax validity and successful execution do not guarantee that a generated rule preserves the intended regulatory meaning.

Having established why natural-language requirements must be converted into formal validation rules, the next subsection introduces the graph-based technologies used in this thesis to represent engineering data and execute such rules.

2.2 Technologies for graph-based validation: RDF and SHACL

A rule can only be checked automatically if the data to be checked is represented in a structured form. Semantic-web technologies provide one way to represent engineering information as linked data. RDF defines an abstract data model for representing information as triples consisting of a subject, predicate, and object [32]. A collection of such triples forms a graph, where entities, properties, and values can be linked explicitly. Turtle is a compact textual syntax for writing RDF graphs [33], while JavaScript Object Notation for Linking Data (JSON-LD) is a JSON-based serialization for linked data [34]. SPARQL, the SPARQL Protocol and RDF Query Language, provides a query language for RDF graphs [35].

In this thesis, the ship design graph is represented as an RDF graph and serialized in Turtle. The generated validation rules are written as SHACL artifacts, meaning that the rules are expressed using the Shapes Constraint Language (SHACL) and serialized in Turtle. This choice allows both the ship data and the generated constraints to be handled using the same RDF-based technology stack.

The Web Ontology Language (OWL) is relevant as an ontology language for describing classes, properties, and semantic relationships [36]. However, OWL and SHACL serve different roles in this thesis. OWL is mainly associated with ontology modeling and logical entailment, whereas SHACL is designed to validate whether an RDF graph satisfies explicitly declared constraints. This distinction is important

because the goal of this thesis is not only to describe ship-domain concepts, but to execute concrete validation rules against a ship design graph.

SHACL is a W3C recommendation for validating RDF graphs [37]. In SHACL terminology, the data being checked is the data graph, while the constraints are expressed in a shapes graph. A SHACL shape defines which nodes are targeted and what conditions those nodes must satisfy. The validation process produces a report that states whether the data graph conforms to the shapes and, if not, where violations occur [37]. SHACL also supports SPARQL-based constraints, which are useful when validation requires graph patterns, conditional logic, arithmetic expressions, or comparisons that are difficult to express with simple property constraints.

SHACL has been studied both as a practical validation language and as a formal object of study. Bogaerts et al. analyze SHACL through the lens of description logic, clarifying the relationship between SHACL constraints and ontology reasoning [38]. Pareti and Konstantinidis provide a broader review of SHACL, covering its validation semantics, reasoning-related problems, and practical adoption [39]. These studies are relevant because they show that SHACL is not merely a syntax for writing checks, but a constraint language with a growing body of formal and practical research behind it.

Graph-based validation has already been applied in adjacent engineering domains. Semantic web technologies have been widely explored in architecture, engineering, and construction for interoperability, linked data, reasoning, and rule checking [40]. Nuyts et al. use SHACL to validate building models against legislation, showing how legal or regulatory conditions can be represented as graph constraints when the relevant data is available in RDF form [41]. Robaldo and Adebayo study compliance checking by translating reified input/output logic into SHACL shapes, demonstrating another connection between normative logic and RDF validation [42]. These works support the use of SHACL as a practical compliance-checking technology, while also showing that the quality of the underlying data model remains critical.

The tooling used in this thesis follows this graph-based validation approach. RDFLib is used for parsing Turtle serializations of RDF and SHACL artifacts and for working with RDF graphs in Python [43]. pySHACL is used to execute SHACL validation against the RDF ship graph [44]. These tools make it possible to evaluate generated rules as executable artifacts rather than only as text. A generated rule can be parsed as Turtle, loaded as a shapes graph, executed against the ship data graph, and inspected through the resulting validation report.

At the same time, a SHACL rule is only meaningful if it uses the correct vocabulary and graph paths for the target data graph. A rule may be valid SHACL but still fail to check the intended data if it targets the wrong node, uses a nonexistent property, or binds to the wrong variable. This is why graph alignment is a central issue in this thesis. The generated rule must not only be valid SHACL; it must also match the structure and vocabulary of the RDF ship design graph. This need for exact graph identifiers is closely related to schema and ontology alignment. Traditional ontology-alignment systems often rely on string-similarity metrics, but such methods can be fragile when equivalent engineering concepts are expressed through abbreviations, supplier-specific labels, unit-bearing names, or subsystem-specific terminology [45]. Recent work has therefore explored large language models for ontology alignment,

using labels and structural context to identify likely equivalence mappings between concepts [46]. In simulation workflows, Rindarøy et al. similarly show that adding higher-level semantics to Functional Mock-up Units can make co-simulation interface connections easier, faster, and more robust [47]. AnchorMap addresses the same issue at the variable-label level by mapping heterogeneous OEM, simulation, and regulatory labels to canonical JSON-LD identifiers with confidence-based routing and human review [7]. These works support the premise that generated SHACL rules need more than regulatory text: they need access to the actual identifiers and graph-access patterns used in the target engineering data.

After introducing RDF and SHACL as the data and validation technologies, the next question is how the SHACL rules themselves can be authored. The following subsection reviews manual, template-based, ontology-derived, and LLM-assisted approaches to rule authoring.

2.3 Rule authoring approaches: manual, template-based, and LLM-assisted

The most direct way to create validation rules is manual authoring by domain and semantic-web experts. Manual authoring can produce high-quality constraints because experts can interpret the regulation, select the relevant data, and encode the rule carefully. However, manual rule authoring is time-consuming and difficult to scale. It requires both domain knowledge and knowledge of the target validation language. It also becomes cumbersome when regulations contain many clauses, when the data model evolves, or when similar rules must be maintained across multiple design variants.

Earlier automated compliance-checking systems often relied on manually encoded rules or domain-specific rule engines [10, 11]. Such systems can work well when the data model and rule set are stable, but they require substantial upfront modeling effort. They also tend to be difficult to adapt when regulations change or when the input model differs from the assumed structure. Consequently, previous research has studied methods for extracting, generating, or semi-automating rules from regulatory text.

Template-based and NLP-based approaches provide one route. These methods typically identify recurring linguistic structures and map them to predefined rule patterns. They may use dependency parsing, semantic role labeling, information extraction, domain ontologies, or manually designed templates. Zhang and El-Gohary’s semantic NLP work is an example of extracting compliance-related information from regulatory documents [12]. More general legal NLP work, such as LexNLP and semantic legal metadata extraction, also demonstrates the importance of detecting structured elements such as obligations, definitions, references, quantities, actors, and conditions in legal or regulatory text [19, 20]. CODE-ACCORD extends this direction by providing annotated regulatory sentences to support information extraction and rule generation research [21].

Natural-language-to-SHACL work is especially close to this thesis. Donkers and

Petrova convert fire safety regulations into SHACL shapes using linguistic analysis and predefined SHACL templates [13]. Their converter uses NLP analysis to identify sentence structure and populate SHACL templates, and the resulting shapes are tested with pySHACL [13]. This work is important because it directly studies the translation from regulatory text into SHACL. At the same time, template-based methods depend on the rule structures anticipated by the template designer. They are strongest when regulations contain regular sentence patterns and comparatively simple property/value conditions, but they become harder to extend when a regulation uses a structure that has not been captured by an existing template.

Another approach is to derive SHACL constraints from existing ontologies or knowledge graphs. Cimmino et al. introduce Astrea, a system for automatically generating SHACL shapes from ontologies [14]. Such methods are valuable when the relevant classes, properties, and schema constraints are already encoded in the ontology or graph structure. In the setting of this thesis, ReqTLo and the ship graph provide part of such a semantic foundation, so ontology-derived shape generation can be seen as a complementary direction rather than a competing one. However, the selected TRAFICOM clauses also contain prescriptive thresholds, formulae, exceptions, and applicability conditions that are not fully encoded as ontology axioms alone. For such cases, the regulatory clause still has to be interpreted and mapped to a validation rule that uses the correct ship-graph identifiers and value paths.

Large language models have created new possibilities for translating natural language into formal artifacts. Yang et al. study translation from natural language to first-order logic (FOL) and present models and datasets for this task [48]. Their work shows that LLMs can support formal translation tasks, but it also highlights that accurate translation from natural language into logic remains challenging. In engineering-data settings, Randles et al. use ChatGPT as part of a linked-data mapping workflow that generates SHACL shapes, SPARQL queries, and RDF sample data from mapping-related context [49]. Westermann et al. present an LLM-assisted pipeline for generating SHACL constraints from textual AutomationML constraints and validating them against an OWL/RDF representation of the model [15]. These works indicate that LLMs can assist with formal and semantic-web artifact generation, but they also show that post-processing and validation are necessary.

For regulatory validation, LLM-assisted rule generation cannot be treated as ordinary text generation. The output must satisfy the syntax of the target language, use the correct vocabulary, target the correct data, and represent the intended requirement. A small mistake in an operator, threshold, unit, applicability condition, or RDF path can change the meaning of the generated rule. This motivates grounding and verification mechanisms, which are discussed in the next subsection.

2.4 State of the art: grounding and verification-guided generation

LLM-generated formal artifacts are useful only when they are both syntactically valid and semantically aligned with the intended task. This is a stricter requirement

than fluent natural-language generation. Hallucination is a known problem in neural text generation: generated output can appear plausible while being unsupported, unfaithful, or incorrect [50]. Benchmarks for language-model truthfulness further support this concern. TruthfulQA shows that language models can generate fluent answers that imitate common falsehoods rather than reliably producing truthful responses [51]. Although TruthfulQA concerns factual question answering rather than SHACL generation, the same surface-plausibility problem appears in this thesis: a generated rule may look technically credible while still being unsupported by the target graph or unfaithful to the source requirement. In formal artifact generation, this risk is especially important because an output may look technically convincing while using invalid identifiers, wrong predicates, incorrect logical conditions, or unsupported assumptions. Jha et al. discuss this problem in the context of LLM-generated formal artifacts and use satisfiability solving and counterexamples to guide correction [52].

One common mitigation strategy is grounding the generation process in external knowledge. Retrieval-augmented generation combines a parametric language model with retrieved external context, allowing generation to be conditioned on information outside the model parameters [53]. In the context of this thesis, the same general idea appears as ship-graph grounding: the generator is not asked to invent a validation vocabulary from regulatory text alone, but is given relevant information about the RDF graph, target nodes, variables, and value-access patterns.

Ontology-guided and knowledge-graph-guided generation are closely related to this idea. Mustafa et al. study natural-language-to-Open Digital Rights Language (ODRL) generation and use the ODRL ontology and documentation to guide policy generation [16]. They emphasize that a term may be semantically plausible in natural language but still invalid if it is not part of the official target vocabulary. AgentODRL extends this direction with a multi-agent architecture for ODRL generation, including decomposition, rewriting, validation-based syntax correction, and semantic reflection [54]. These works are relevant because ODRL, like SHACL, is a structured policy or rule language where vocabulary alignment matters.

Similar grounding ideas appear in engineering-model generation. Westermann et al. present a pipeline in which an AutomationML model is mapped to an OWL/RDF representation, textual constraints are translated into SHACL shapes by an LLM, and the generated constraints are validated against the generated ontology [15]. Qualis studies SysML v2 generation using a tri-layered knowledge-graph reasoning framework intended to reduce hallucinated model elements [55]. Vieira da Silva et al. investigate the use of LLMs for generating capability ontologies and evaluate outputs through RDF syntax checking, OWL reasoning, and SHACL constraints [56]. Although these works target different artifacts, they share the same principle: generated formal models must be constrained by the target schema, ontology, or knowledge graph.

Grounding addresses vocabulary and schema alignment, but it does not guarantee correctness. This has motivated feedback and verification-guided approaches, where a generated artifact is checked and the feedback is used to revise the next attempt. Self-Refine demonstrates a general iterative pattern in which an LLM generates an initial output, produces feedback, and refines the output without additional training [57]. Tandon et al. study repair after deployment using a dynamic memory of feedback,

showing how past feedback can support later corrections [58]. Han et al. present PiVe, an iterative verification framework for graph-based generation that uses verifier feedback to improve generated semantic graphs [59]. These approaches support the idea that generation and assessment should be separated, and that feedback can improve structured outputs.

However, verification-guided repair is not automatically sufficient. The quality of the final artifact depends on the quality of the verifier, the specificity of feedback, and the generator’s ability to use that feedback without introducing new errors. In SHACL generation, deterministic tool checks are helpful because they provide concrete evidence: RDFLib can report Turtle syntax errors, and pySHACL can report execution errors and validation results. Nevertheless, these checks do not prove that the generated rule represents the intended regulation. A rule can be syntactically valid and executable while still targeting the wrong graph node or checking an incomplete condition.

Overall, the reviewed literature points toward three design principles for LLM-assisted generation of formal validation rules:

1. **Ground the generated artifact in the target model.** Generated rules should be conditioned on the relevant vocabulary, ontology, schema, or data graph rather than produced from natural-language text alone.
2. **Check generated artifacts with deterministic tools.** Syntax checking, graph parsing, and SHACL execution provide concrete evidence about whether the generated artifact can be used as an executable validation rule.
3. **Use feedback for repair, but evaluate the repaired artifact empirically.** Feedback from checking can help repair generated artifacts, but repair can also fail to detect semantic errors or introduce new ones, so final correctness still has to be evaluated in the target engineering setting.

These principles motivate the generation–execution–repair design evaluated in this thesis.

2.5 Research gap and positioning of this thesis

The reviewed work establishes the technical foundations for this thesis. Automated compliance checking research shows that natural-language regulations must be transformed into machine-interpretable rules before they can be checked automatically [10, 11]. Semantic-web technologies provide a graph-based representation and validation stack through RDF, Turtle, SPARQL, OWL, JSON-LD, and SHACL [32–37]. Natural-language-to-rule and natural-language-to-SHACL research shows that template-based and NLP-based methods can convert some regulatory clauses into validation artifacts [12, 13]. LLM and knowledge-graph research shows that formal artifact generation benefits from grounding, execution checks, and feedback-guided repair [15, 16, 54, 55, 59].

The remaining gap is the combination of these ideas for LLM-assisted SHACL generation in an engineering validation setting where the generated rule must work against a concrete RDF data model. This setting is stricter than ordinary text generation. A generated rule must be valid SHACL, must run in a SHACL validator, must target the correct data, and must represent the intended regulatory requirement closely enough to support engineering review. A rule that only looks correct is not sufficient.

The gap can be summarized across five research streams:

1. **Automated compliance checking.** This stream explains why regulations must be transformed into computable rules, but it often assumes manually encoded rules or domain-specific checking infrastructure [10, 11].
2. **Semantic-web validation.** RDF and SHACL provide executable checking over graph data, but this stream usually assumes that the SHACL shapes already exist or can be generated from known schemas [14, 37, 41].
3. **NLP and template-based rule extraction.** This stream provides controlled conversion from regulatory text to predefined rule patterns, but it is limited when clauses contain varied wording, formulae, exceptions, and domain-specific graph paths [12, 13].
4. **LLM-based formal artifact generation.** LLMs provide more flexible translation from natural language to formal artifacts, but they introduce risks of hallucination, schema mismatch, and plausible but wrong outputs [15, 16, 48].
5. **Verification-guided generation.** Tool feedback, verifier feedback, and repair loops can improve generated artifacts, but they still need empirical evaluation in the target engineering setting because feedback can filter outputs, improve them, or fail to detect semantic errors [52, 54, 59].

No single stream fully addresses the setting evaluated in this thesis: translating selected maritime regulatory requirements into executable SHACL constraints, running them against a concrete RDF ship graph, using execution feedback for repair, and manually assessing whether the generated rule checks the intended requirement for the correct reason.

This thesis addresses this gap by developing and evaluating NLTL, a pipeline that translates selected natural-language maritime regulations into SHACL constraints and checks them against an RDF ship graph. The evaluation focuses on whether LLMs can generate SHACL rules that can be run by validation tools, how generated artifacts can be checked for syntax, SHACL execution, and graph alignment, and how ship-graph grounding and validator-guided repair affect correctness and limitations.

The contribution of this thesis is the design and empirical evaluation of an LLM-assisted SHACL rule-generation pipeline in a maritime engineering context. The work studies the rule-authoring bottleneck that remains after an RDF-based validation infrastructure exists: converting selected regulatory text into executable SHACL artifacts that are aligned with the RDF ship graph and suitable for expert review. This

connects natural-language regulation, ship-graph grounding, deterministic validation tools, validator-guided repair, and manual semantic assessment in one evaluated workflow.

3 Materials and methods

This section describes the materials, implementation, and evaluation procedure used to develop and assess the NLTL pipeline. The section first presents NLTL as the designed method: a generation–execution–repair workflow for translating structured regulatory cases into executable SHACL constraints. It then describes the maritime case-study setting used to instantiate and evaluate the method, including the selected TRAFICOM requirements, the RDF ship design graph, and the structured case representation. Finally, the section defines the evaluation procedure, ablation configurations, and repeated-run summary statistics.

The method is not treated as a pure text-generation task. In the full NLTL pipeline, each experimental unit consists of one structured regulatory case, one generated SHACL artifact, deterministic syntax and execution evidence, and one validator-guided decision. In the ablation configurations, the same case-level structure is used, but the validator decision and repair loop are removed as described in Section 3.3. The larger case-study experiment reported in Section 4 repeats this single-case procedure across the benchmark set.

3.1 NLTL pipeline

This thesis follows a Design Science Research (DSR) approach. The designed artifact is NLTL, an LLM-assisted pipeline for translating natural-language regulatory requirements into executable SHACL constraints. The pipeline was developed through an iterative design–build–evaluate process: selected requirements were represented as structured cases, a single-case generation and execution workflow was implemented, validator-guided repair was added, and the resulting workflow was evaluated on a benchmark set of regulatory cases [60, 61]. The implementation used for the experiments is available as a public GitHub repository [62]. The central methodological principle is that a generated SHACL artifact is not judged only by whether the tested data graph conforms or does not conform. A candidate must be syntactically valid, executable, aligned with the target RDF data model, and must represent the intended regulatory requirement correctly. This distinction is necessary because an incorrect rule may execute successfully while checking the wrong target, wrong path, wrong variable, or wrong regulatory condition.

Figure 2 presents the single-case NLTL workflow. The pipeline combines deterministic software components with two LLM-based stages. The deterministic components perform file loading, SHACL extraction, Turtle syntax checking, SHACL execution, error compilation, artifact storage, and iteration control. The LLM-based stages perform generation and validation: the generator model proposes a SHACL candidate, while the validator model assesses whether the candidate should be accepted, repaired, or rejected.

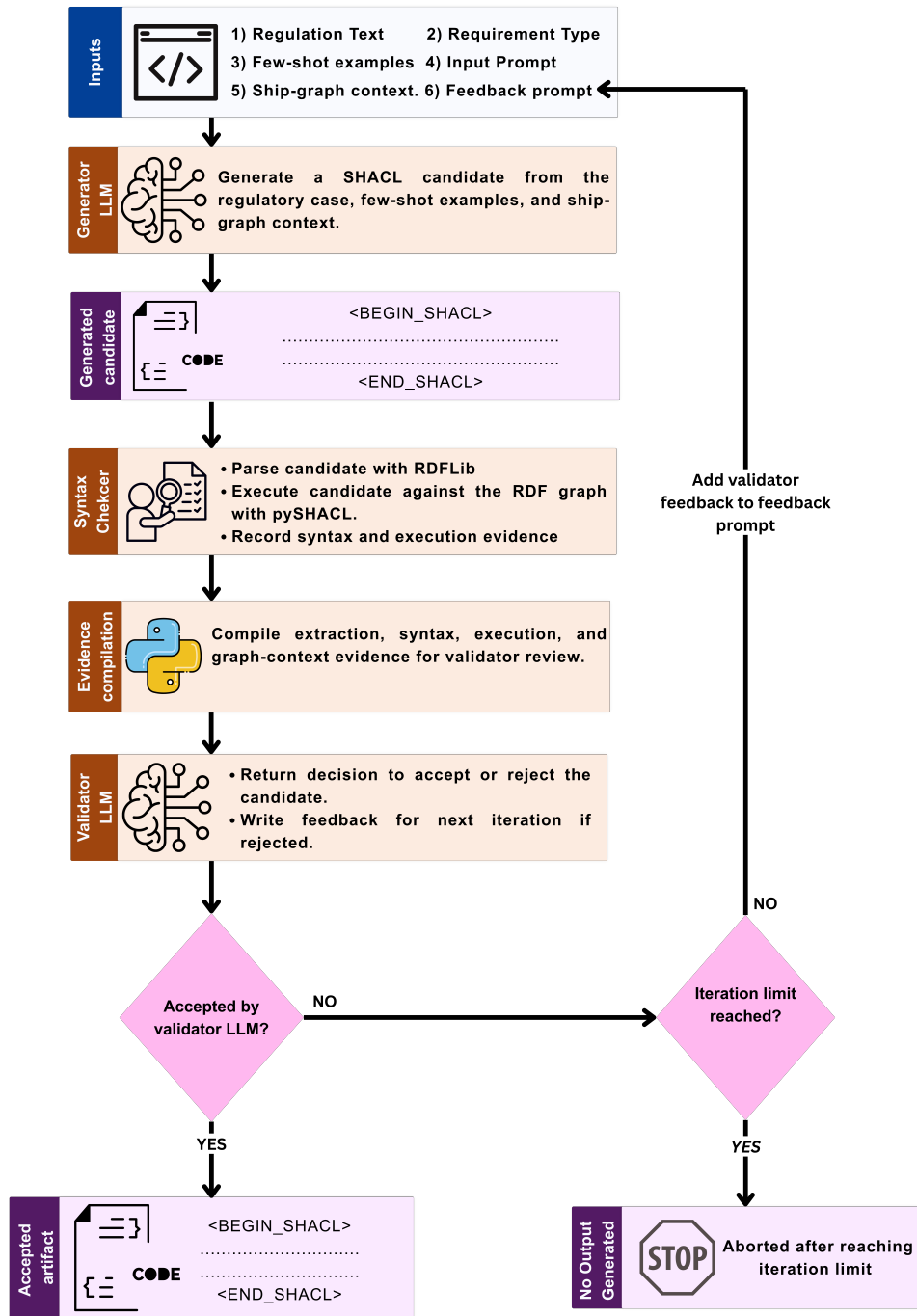


Figure 2: Overview of the NLTL single-case generation-execution-repair workflow. The generator produces a SHACL candidate from the regulatory case, ship-graph context, and any previous feedback. The candidate is checked with RDFLib and pySHACL, assessed by the validator model, and either accepted, repaired through feedback, or aborted after the iteration limit.

Inputs. For each case, the input stage collects the information used to construct the generation prompt. This includes the regulation text, requirement type, few-shot examples, input prompt, ship-graph context, and, when applicable, feedback from the previous failed attempt. The requirement type is used to select the most relevant few-shot pattern for the case, while the ship-graph context provides the target RDF vocabulary, relevant nodes, variables, and value-access paths. This input structure is intended to reduce ambiguity before generation by giving the model both the regulatory clause and the data-model context in which the generated SHACL must execute. The prompt templates used in the experiments are provided in Appendix D. For the full NTL pipeline, the generator prompt is shown in Listing 6 and the validator prompt is shown in Listing 7. The one-shot generator prompts used for the two ablation configurations are shown separately in Listings 8 and 9.

Generator LLM. The generator LLM produces a candidate SHACL artifact from the structured regulatory case and the available context. The prompt instructs the generator to declare all prefixes that it uses, follow the vocabulary and graph-access patterns provided in the case context, and avoid inventing class names, property names, or variable names when corresponding identifiers are available in the target data graph. For simple value checks, direct SHACL property constraints may be sufficient. For requirements involving numeric comparisons, conditional applicability, arithmetic expressions, formula-related checks, or multiple graph bindings, the generator is guided to use SHACL-SPARQL. This guidance is intended to make the generated rule executable against the intended RDF data model rather than merely plausible as text.

Generated candidate. The generated candidate is the raw SHACL block returned by the generator. The pipeline extracts the text between <BEGIN_SHACL> and <END_SHACL>. If either marker is missing, duplicated in an invalid way, or ordered incorrectly, the candidate is marked as malformed and the extraction error is recorded for the next repair iteration. When a valid block is found, only the enclosed SHACL text is retained. This extracted artifact becomes the candidate shapes graph for the current iteration.

Syntax checker. The syntax checker first parses the extracted candidate as Turtle using RDFLib [43]. This step checks deterministic Turtle syntax validity before SHACL validation is attempted. If RDFLib parsing fails, the parser error is recorded as syntax feedback and the candidate cannot proceed to execution on the ship graph in that iteration. If the candidate is syntactically valid, the parsed SHACL artifact is executed against the target RDF ship graph using pySHACL [44]. pySHACL returns the conformance result, validation report, and any execution errors.

Evidence compilation. The evidence compilation stage collects the deterministic evidence produced by extraction, RDFLib parsing, and pySHACL execution. The compiled evidence includes extraction status, syntax validity, syntax errors, execution status, conformance result, validation report information, and execution errors when present. This compact evidence package is passed to the validator model together with the generated SHACL and the original case context. The purpose is to make the validator decision depend not only on the generated text, but also on how the candidate behaved when parsed and executed.

Validator LLM. The validator LLM assesses whether the generated SHACL is

faithful to the source requirement, aligned with the RDF vocabulary and value-access patterns, and responsible for the observed validation outcome for the correct reason. It receives the structured regulation case, generated SHACL, syntax evidence, SHACL execution evidence, data-graph context, and few-shot examples. The validator returns a structured decision containing an acceptance status, expected outcome, actual outcome, identified issues, and suggested repair. This stage separates generation from assessment, so the model that produces the rule is not the only mechanism judging whether the rule should be accepted.

Accepted by validator LLM and iteration limit. A generated SHACL artifact is accepted only when four conditions are satisfied: the SHACL block is extracted successfully, the extracted Turtle is syntactically valid, pySHACL execution succeeds, and the validator decision is `accept`. If any condition is not satisfied, the pipeline checks whether the maximum iteration limit has been reached. When iterations remain, syntax errors, execution errors, and validator comments are passed back to the generator as repair feedback. This creates the generation–execution–repair loop shown in Figure 2.

Accepted artifact and no-output cases. If the validator accepts a candidate before the iteration limit is reached, the extracted SHACL is stored as the accepted artifact for that case. If the iteration limit is reached without an accepted candidate, the case is marked as aborted and no accepted SHACL output is produced. The pipeline stores all artifacts needed for later inspection and traceability, including the raw generator response, extracted SHACL candidate, extraction result, RDFLib syntax result, pySHACL execution report, compiled error summary, validator response, repair feedback, and final case decision. This makes it possible to inspect not only whether a case succeeded or failed, but also where in the pipeline the failure occurred.

3.2 Experimental material and case representation

The experimental material consists of selected regulatory requirements from the Finnish Transport and Communications Agency regulation *Ice Class Regulations and the Application Thereof* (TRAFICOM/68863/03.04.01.00/2021), issued on 1 July 2021 and in force from 5 July 2021 [8]. The thesis does not attempt to formalize the complete regulation. Instead, individual clauses or clause fragments were selected when they could be represented as distinct validation tasks against the available RDF ship design graph.

The final benchmark consisted of 90 regulatory cases. The natural-language regulatory inputs used in the benchmark are listed in Appendix B. Each case was represented as one structured input to the pipeline and was assigned to one of four operational categories. These categories were used for prompt construction, few-shot selection, result aggregation, and failure analysis. Table 2 summarizes the requirement categories used in the benchmark dataset.

The generated SHACL constraints were executed against a ship design graph represented as RDF and serialized in Turtle. The graph acted as the target data model for validation and defined the vocabulary that generated rules had to use. It contains a main ship system node, component nodes, and engineering variable nodes.

Table 2: Requirement categories used in the benchmark dataset.

Category	Number of cases	Description
Static	24	Directly verifiable requirements such as thresholds, enumeration checks, lookup values, or simple bounded constraints. These cases can usually be checked by comparing a value in the ship graph against a fixed regulatory condition.
Static calculation	16	Requirements involving formula-based or computed-value comparisons. In these cases, the generated SHACL may either compute a simple expression in SPARQL or compare a stored or precomputed value against the regulatory limit.
Complex readiness	42	Requirements where the validation task checks whether the ship graph contains the variables, outputs, or structural information needed for later compliance verification. These cases often correspond to formulas or engineering checks that are too complex to reconstruct reliably inside SHACL-SPARQL alone.
Dynamic readiness	8	Propulsion, response, vibration, fatigue, or analysis-dependent requirements. Full physical or time-domain simulation is outside the scope of this thesis, so these cases are treated as simulation-readiness or required-output checks rather than complete dynamic compliance verification.

Many engineering quantities are represented as variable nodes rather than as direct datatype properties of a ship or component. The usual value-access pattern in the graph is shown in Listing 1. A variable node is associated with a ship or component using `ssn:isPropertyOf`, and the numerical, Boolean, string, or date value is then read through `ssp:hasVariableValue`. A generated rule therefore has to follow the implemented RDF graph structure rather than only the surface wording of the regulation.

```
?variable ssn:isPropertyOf ?component ;
        ssp:hasVariableValue ?value .
```

Listing 1: RDF value-access pattern used for engineering quantities in the ship design graph.

Each selected requirement was represented as a structured JSON case file. The case file acted as a lightweight intermediate representation between the source regulation and the generated SHACL artifact. It did not formalize the full logic of the regulation by itself; rather, it stored the information needed to construct prompts, select few-shot examples, execute the generated rule, and trace the output back to the original case.

Table 3: Main information represented in each structured regulation case file.

Information type	Purpose in the pipeline
Case identity	Identifies the target case and links generated outputs to the corresponding benchmark item.
Regulatory content	Stores the source clause, paraphrased rule, or concise case statement to be translated into SHACL.
Requirement classification	Records the requirement category and type used for few-shot selection and result aggregation.
Validation context	Stores information needed to run or interpret the generated SHACL against the ship graph, such as the relevant ship file or expected validation behaviour.
Experiment metadata	Links generated artifacts, validator decisions, execution results, and manual evaluation outcomes to the original case.

Table 3 summarizes the main information stored in each case file.

The case representation supports both prompting and traceability. Because every generated SHACL artifact is linked to a case identifier, the experiment runner can store the raw generator response, extracted SHACL, syntax result, pySHACL report, validator response, and final decision under the same case context. This makes it possible to evaluate outputs across requirement categories, repeated runs, and ablation configurations.

3.3 Evaluation procedure

The evaluation examined whether each pipeline configuration produced SHACL artifacts that were executable and semantically correct for the intended regulatory case. A produced output was counted only when a configuration generated a SHACL artifact for the case. For the full NLTL pipeline, this meant that the candidate satisfied the acceptance criteria before the iteration limit was reached. For the two ablation configurations without validator-guided repair, this meant that the generated SHACL artifact was treated as the produced output for the case.

Correctness was assessed manually after execution. A generated SHACL artifact was marked correct only when it represented the intended regulatory requirement, used the correct target or component, followed the RDF vocabulary and value-access pattern of the ship graph, encoded the intended comparison or readiness check, and produced the observed ship outcome for the correct reason. Syntax validity and pySHACL executability were therefore necessary evidence, but not sufficient evidence, for correctness.

Three quantitative metrics were used. In the metric definitions, N_{correct} always refers to the number of produced SHACL artifacts that were manually judged semantically correct after execution. The output rate measures how often a configuration produced an output:

$$\text{OutputRate} = \frac{N_{\text{produced}}}{N_{\text{total}}}$$

where N_{produced} is the number of cases for which an output was produced and N_{total} is the total number of benchmark cases.

Accuracy among produced outputs measures correctness only among the cases for which an output was produced:

$$\text{AccuracyProduced} = \frac{N_{\text{correct}}}{N_{\text{produced}}}$$

where N_{correct} is the number of produced outputs manually judged semantically correct. This metric describes the precision of the produced artifacts and is especially relevant for the full pipeline, which may abort cases that do not satisfy the acceptance criteria.

Overall accuracy measures correctness over the full benchmark set:

$$\text{OverallAccuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}}$$

This metric penalizes both outputs manually judged incorrect and cases where no output was produced. The distinction between accuracy among produced outputs and overall accuracy is important because a selective pipeline may produce fewer outputs with higher precision, while a configuration without validator-guided repair may produce outputs for all cases but with lower correctness. Each evaluated configuration was run three times on the same benchmark set. The repeated runs were used to describe run-to-run variability in the aggregate metrics. For each configuration, the thesis reports the mean and sample standard deviation across the three runs; for the percentage-based metrics, the standard deviations are interpreted as percentage points. Because each produced SHACL artifact required manual semantic correctness assessment, the number of repeated runs was limited to three. Welch’s unequal-variance t-tests were used as supplementary exploratory checks for selected pairwise comparisons between configurations [63, 64]. These tests were used only as supporting evidence, because the small number of repeated runs means that the estimated standard deviations remain uncertain.

In addition to the quantitative metrics, incorrect produced outputs were assigned failure modes during manual analysis. The final categories were wrong RDF path or target, syntax or execution problem, incomplete validation logic, applicability or edge-case gap, formula/table calculation problem, and other. These categories were used to support the discussion of why the pipeline succeeded or failed across configurations and repeated runs.

To evaluate the contribution of the main pipeline components, the full NLTL pipeline was compared with two ablation configurations. The purpose of the ablation study was to separate the effect of ship-graph context from the effect of validator-guided repair. The full configuration follows the complete generation–execution–repair workflow shown in Figure 2. The two ablation workflows are shown in Figures 3 and 4.

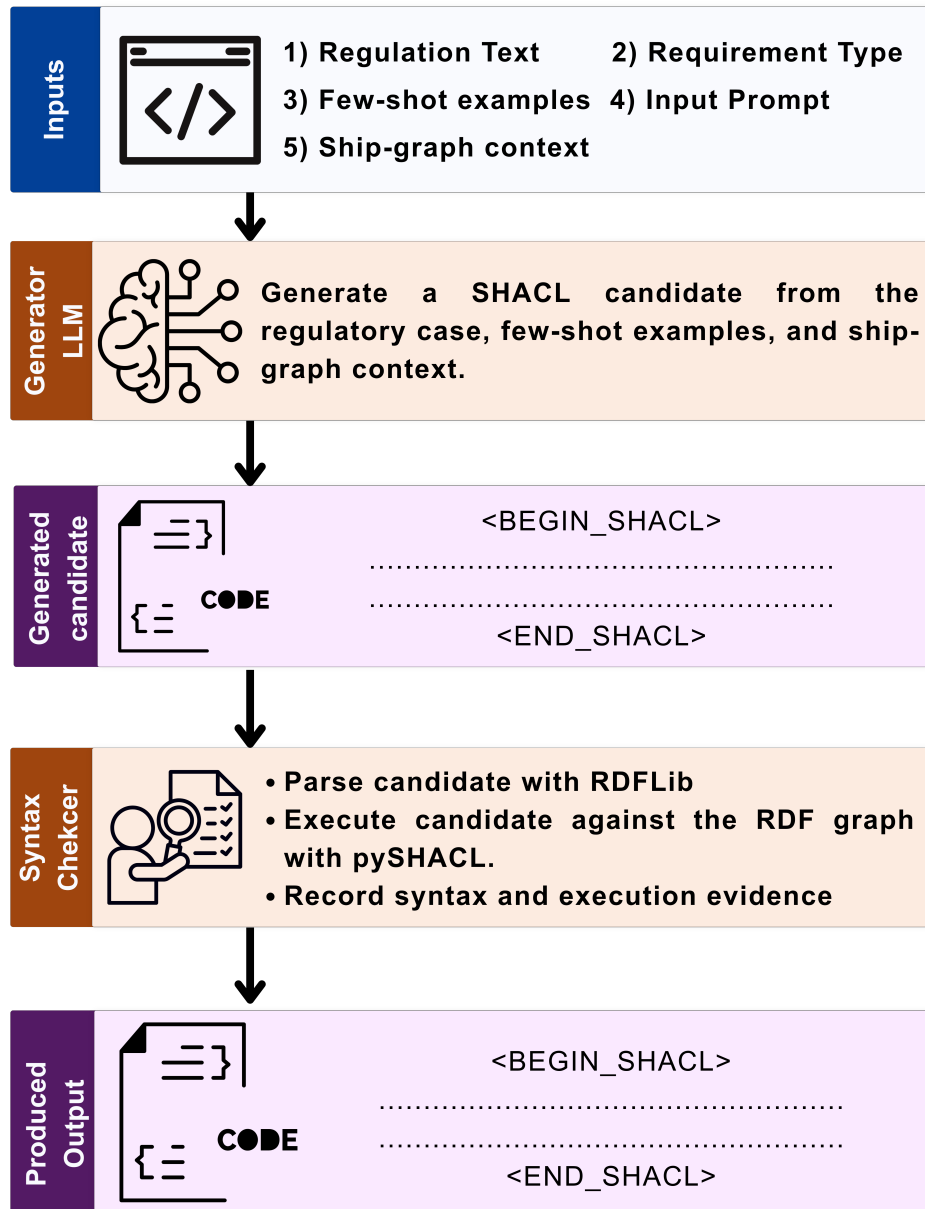


Figure 3: Workflow for the NLTL without validator configuration. The generator uses ship-graph context, but validator-guided acceptance and repair are removed. The generated SHACL artifact is checked and then evaluated directly.

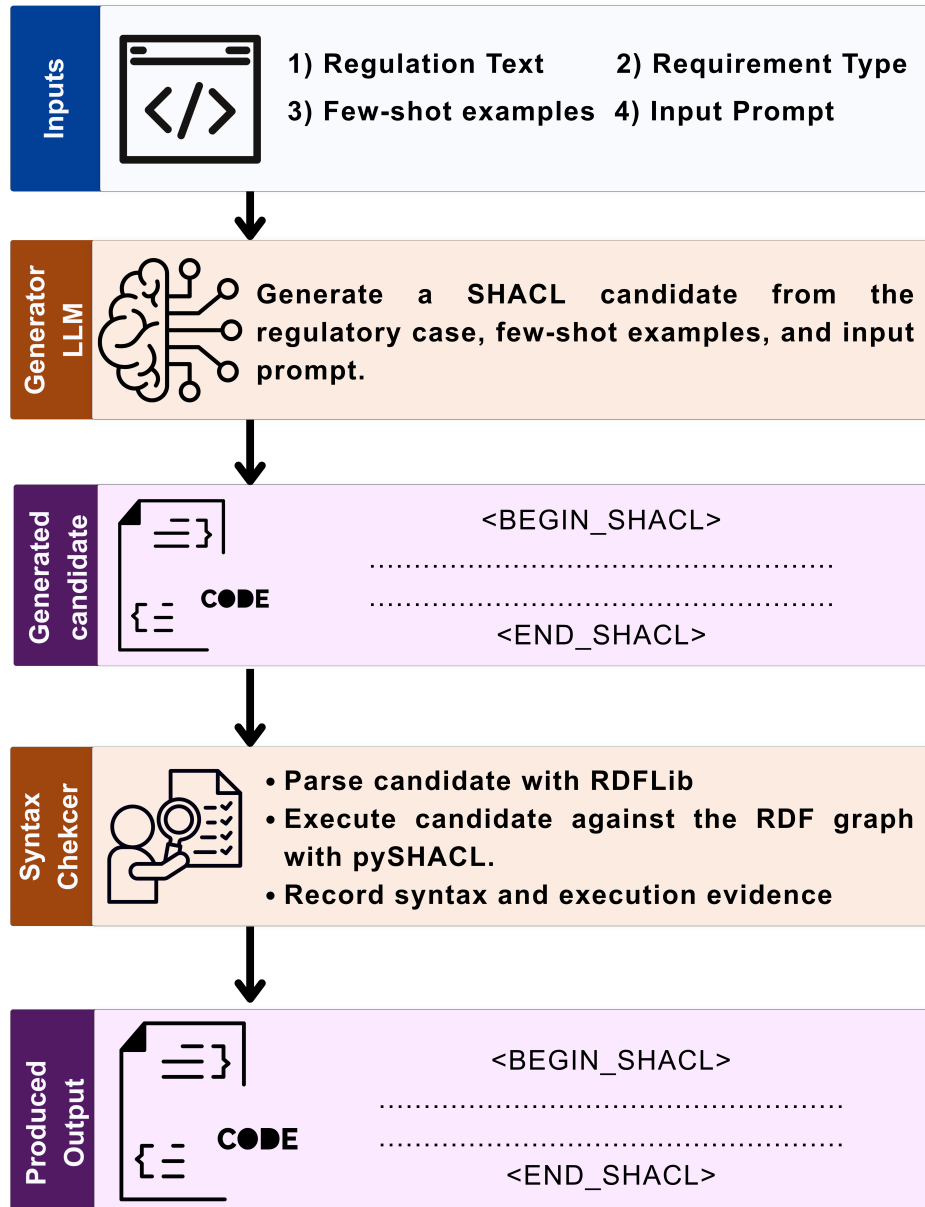


Figure 4: Workflow for the NLTL without validator and without ship context configuration. The generator uses only the regulation case, requirement type, few-shot examples, and input prompt. The generated SHACL artifact is checked and then evaluated directly.

Table 4 summarizes the same three configurations in terms of the workflow components that were kept, removed, or changed.

Table 4: Ablation configurations mapped to the NLTL workflow in Figure 2.

Configuration	Workflow used	Removed or changed component
Full NLTL pipeline	Complete generation–execution–repair workflow.	No component is changed.
NLTL without validator	Generator uses the regulation case, requirement type, few-shot examples, input prompt, and ship-graph context. The generated SHACL artifact is evaluated directly.	Removes validator acceptance, validator feedback, and iterative repair.
NLTL without validator and without ship context	Generator uses only the regulation case, requirement type, few-shot examples, and input prompt. The generated SHACL artifact is evaluated directly.	Removes both ship-graph context and validator-guided repair.

The comparison between these configurations evaluates two effects. Comparing NLTL without validator and without ship context with NLTL without validator shows how much ship-graph context contributes to vocabulary alignment, target selection, and RDF path construction. Comparing NLTL without validator with the full NLTL pipeline shows how validator-guided feedback and repair affect the correctness and selectivity of accepted outputs. The full pipeline may produce fewer outputs because it can abort when no candidate satisfies the acceptance criteria, but this selectivity is part of the evaluated method. For both ablation configurations, generated artifacts were still checked and manually assessed during evaluation; what is removed is the use of validator feedback as an iterative repair and acceptance mechanism. The evaluated configurations also differed in their prompt structure. The full NLTL pipeline used the graph-grounded generator prompt in Listing 6 and the validator prompt in Listing 7. NLTL without validator used the one-shot graph-grounded ablation prompt in Listing 8, where validator-based acceptance, feedback, and repair were removed. NLTL without validator and without ship context used the one-shot no-context ablation prompt in Listing 9, where both ship-graph context and validator-guided repair were removed. The prompt templates are listed in Appendix D.

The outputs of these three configurations form the empirical basis for the case-study results reported in Section 4. The implications of the evaluation design, including manual correctness assessment, run-to-run variability, validator uncertainty, and case-study boundaries, are discussed later in Section 5.3.

4 Case study and evaluation

Section 3 described NLTL as a single-case generation–execution–repair workflow. This section evaluates that workflow through a maritime case study based on the Finnish Transport and Communications Agency (TRAFICOM) ice-class regulation. The purpose of the case study is to report how the evaluated configurations performed when translating selected natural-language regulatory requirements into executable SHACL constraints for an RDF ship design graph.

In practical terms, the case study tests three questions: whether the pipeline can produce executable SHACL artifacts, whether those artifacts are semantically correct for the selected regulatory cases, and how the results change when ship-graph context or validator-guided repair is removed.

The TRAFICOM ice-class regulation was used because it provides a realistic and technically varied regulatory source. It contains direct value checks, formula-related requirements, table-based coefficients, applicability conditions, required engineering variables, and analysis-dependent requirements [8]. This makes it suitable for evaluating whether generated SHACL rules can handle both simple constraints and cases where the validation logic depends on the structure and vocabulary of the ship graph.

The case study builds on earlier work on ontology-enabled maritime engineering validation and variable standardisation. ReqTLo showed how formal requirement models and SHACL-based checks can support simulation-linked validation of engineering requirements [9]. AnchorMap addressed a complementary upstream problem by standardising heterogeneous engineering variable labels to canonical JSON-LD identifiers so that downstream co-simulation and compliance workflows can operate on shared vocabulary [7]. In both cases, the availability of structured identifiers is central. The present thesis evaluates the next step: whether selected natural-language regulatory requirements can be translated into executable SHACL constraints with the help of LLM-based generation, ship-graph grounding, deterministic validation tools, and validator-guided repair.

The evaluation uses 90 TRAFICOM-derived regulatory cases. Each case represents one selected requirement or requirement fragment and links it to an expected validation behaviour over an RDF ship design graph. The cases cover static requirements, static calculation requirements, complex readiness requirements, and dynamic readiness requirements. These categories describe the benchmark composition, but the main results in this chapter are reported at the configuration level.

The three configurations defined in Table 4 were each executed three times on the same set of 90 cases. The repeated executions are referred to as Run 1, Run 2, and Run 3. After execution, each produced SHACL artifact was manually checked for semantic correctness. A produced artifact was marked correct only if it represented the intended regulatory requirement, used the correct RDF target or component, followed the ship-graph vocabulary and value-access pattern, and produced the expected validation behaviour for the correct reason. The reported results therefore distinguish between output production, syntactic executability, and manual semantic correctness.

The remainder of this section is organized as follows. Section 4.1 describes the

case-study setup and experimental configurations. Section 4.2 reports the repeated-run results for the full NLTL pipeline. Section 4.3 reports the ablation comparison between the three configurations. Section 4.4 reports the manually assigned failure modes for incorrect produced outputs.

4.1 Case-study setup and experimental configurations

The case-study experiment applies the evaluated pipeline configurations to selected requirements from the TRAFICOM ice-class regulation. The regulation was selected because it is a real maritime regulatory document with requirements that vary in structure and difficulty. Some cases involve direct value comparisons, while others involve formula-related logic, table lookup, applicability conditions, or the presence of variables needed for later engineering analysis. Table 5 summarizes the main experimental settings used in the case-study evaluation.

Table 5: Main experimental settings for the case-study evaluation.

Setting	Value
Regulatory source	TRAFICOM <i>Ice Class Regulations and the Application Thereof</i>
Benchmark size	90 regulatory cases
Requirement categories	Static; static calculation; complex readiness; dynamic readiness
Validation target	RDF ship design graph
Generator model	GPT-5.1
Validator model	GPT-5.1
Maximum repair iterations	10 per case for the full NLTL configuration
SHACL extraction markers	<BEGIN_SHACL> and <END_SHACL>
Syntax checking	RDFLib Turtle parser
SHACL execution	pySHACL
SHACL execution settings	RDFS inference enabled; meta-SHACL disabled
Evaluated configurations	Full NLTL pipeline; NLTL without validator; NLTL without validator and without ship context
Repeated runs	Three runs per configuration
Manual assessment	Each produced SHACL artifact was manually checked for semantic correctness
Reported metrics	Produced outputs; correct outputs; incorrect outputs; no-output cases; output rate; accuracy among produced outputs; overall accuracy; mean and sample standard deviation across repeated runs; failure modes

The benchmark consists of 90 structured regulatory cases grouped into static, static calculation, complex readiness, and dynamic readiness categories. Each case

is linked to a regulatory input, an expected validation behaviour, and an RDF ship design graph. The generated SHACL constraints are executed against this ship graph, so the task is not only to produce valid SHACL syntax, but also to align the generated rule with the identifiers and value-access paths used in the RDF data model.

The evaluated configurations are the full NLTL pipeline and the two ablation configurations defined in Table 4. The case-study setup therefore keeps the regulatory benchmark fixed while varying whether the generator receives ship-graph context and whether validator-guided repair is used.

Correctness was assessed manually because successful execution alone does not prove that a generated rule represents the intended regulatory meaning. A generated SHACL artifact may be syntactically valid and executable while still targeting the wrong RDF node, using the wrong graph path, omitting an applicability condition, or checking only part of the intended logic. The manual assessment therefore checked whether the produced artifact followed the ship-graph vocabulary and produced the expected validation behaviour for the correct reason.

The three repeated runs were used to report the stability of the aggregate metrics. For each configuration, individual run outcomes are reported first. Percentage-based summary metrics are then reported as mean $\pm$ sample standard deviation in Table 9, with the deviations expressed in percentage points (pp).

4.2 Full NLTL pipeline repeated-run results

This subsection reports the results of the full NLTL pipeline across three repeated runs. The pipeline was executed three times on the same set of 90 regulatory cases. Each produced SHACL artifact was manually checked for semantic correctness after execution. Table 6 reports the per-run results.

Table 6: Repeated-run results for the full NLTL pipeline. Results are reported over 90 regulatory cases.

Metric	Run 1	Run 2	Run 3
Produced outputs	53	55	52
Correct outputs	37	35	35
Incorrect outputs	16	20	17
No-output cases	37	35	38
Output rate	58.9%	61.1%	57.8%
Accuracy among produced outputs	69.8%	63.6%	67.3%
Overall accuracy	41.1%	38.9%	38.9%

Across the three full NLTL pipeline runs, the number of produced outputs ranged from 52 to 55, while the number of manually correct outputs ranged from 35 to 37.

4.3 Ablation studies

This subsection reports the ablation results for the three configurations defined in Table 4. The comparison is designed to separate two effects: the effect of ship-graph grounding and the effect of validator-guided repair. NLTL without validator tests graph-aware generation without iterative validator feedback. NLTL without validator and without ship context tests generation from the structured regulation case and few-shot examples alone. Each configuration was executed three times on the same set of 90 regulatory cases, and each produced SHACL artifact was manually checked for correctness.

Table 7: Repeated-run results for NLTL without validator. Results are reported over 90 regulatory cases.

Metric	Run 1	Run 2	Run 3
Produced outputs	90	90	90
Correct outputs	52	55	54
Incorrect outputs	38	35	36
No-output cases	0	0	0
Output rate	100.0%	100.0%	100.0%
Accuracy among produced outputs	57.8%	61.1%	60.0%
Overall accuracy	57.8%	61.1%	60.0%

NLTL without validator produced an output for all 90 cases in each run. This 100.0% output rate follows directly from the ablation setting: without validator-guided acceptance and repair, the first generated SHACL artifact was treated as the produced output for each case. Therefore, there were no no-output cases in this configuration, and accuracy among produced outputs was identical to overall accuracy.

Table 8: Repeated-run results for NLTL without validator and without ship context. Results are reported over 90 regulatory cases.

Metric	Run 1	Run 2	Run 3
Produced outputs	90	90	90
Correct outputs	5	9	9
Incorrect outputs	85	81	81
No-output cases	0	0	0
Output rate	100.0%	100.0%	100.0%
Accuracy among produced outputs	5.6%	10.0%	10.0%
Overall accuracy	5.6%	10.0%	10.0%

NLTL without validator and without ship context also produced an output for all 90 cases in each run for the same reason: the ablation configuration did not include

a validator acceptance gate or iteration-limit abort. However, the number of correct outputs was much lower than in the corresponding configuration with ship context.

Table 9: Mean and sample standard deviation of the main evaluation metrics across three repeated runs ($n = 3$). The $\pm$ values are sample standard deviations across runs and are reported in percentage points (pp).

Metric	Full NLTL pipeline	NLTL without validator	NLTL without validator and without ship context
Output rate	59.3% $\pm$ 1.7 pp	100.0% $\pm$ 0.0 pp	100.0% $\pm$ 0.0 pp
Accuracy among produced outputs	66.9% $\pm$ 3.1 pp	59.6% $\pm$ 1.7 pp	8.5% $\pm$ 2.6 pp
Overall accuracy	39.6% $\pm$ 1.3 pp	59.6% $\pm$ 1.7 pp	8.5% $\pm$ 2.6 pp

Across the three full-pipeline runs, the full NLTL configuration produced accepted outputs for 52–55 cases and produced 35–37 correct outputs. NLTL without validator produced outputs for all 90 cases in each run and produced 52–55 correct outputs. NLTL without validator and without ship context also produced outputs for all 90 cases in each run, but only 5–9 of these outputs were correct.

For both ablation configurations, the output rate was 100.0% in all three runs by construction. Therefore, accuracy among produced outputs and overall accuracy are identical for these two configurations. For the full NLTL pipeline, these metrics differ because some cases ended with no accepted output. The largest difference was observed between the configuration with ship context and the configuration without ship context. Removing ship context reduced the mean overall accuracy from 59.6% to 8.5%, while both configurations still produced outputs for all cases. The full NLTL pipeline had lower mean overall accuracy than NLTL without validator because it accepted fewer outputs, but it achieved the highest mean accuracy among produced outputs.

4.4 Failure-mode analysis

Incorrect produced outputs were manually assigned to broad failure-mode categories. Aborted or no-output cases are not included in this analysis because they did not produce a final evaluable SHACL artifact. The failure modes describe where the produced SHACL artifact failed. A wrong RDF path or target means that the rule referred to an incorrect class, node, or property path in the ship graph. A syntax or execution problem means that the generated artifact could not be parsed or executed reliably as SHACL/SPARQL. Incomplete logic means that the generated rule executed but did not fully encode the regulatory condition. Applicability gaps refer to missing or incorrect handling of conditional scope, exceptions, or edge cases. Formula or

table errors refer to incorrect translation of numerical formulae, lookup-table logic, or regulatory coefficient logic.

Figures 5–7 report the failure-mode distributions across the three repeated runs for each evaluated configuration. Each stacked bar represents the incorrect produced outputs in one run, decomposed by failure mode. The number above each bar gives the number of incorrect produced outputs, while N_{produced} below each bar gives the total number of produced outputs in that run.

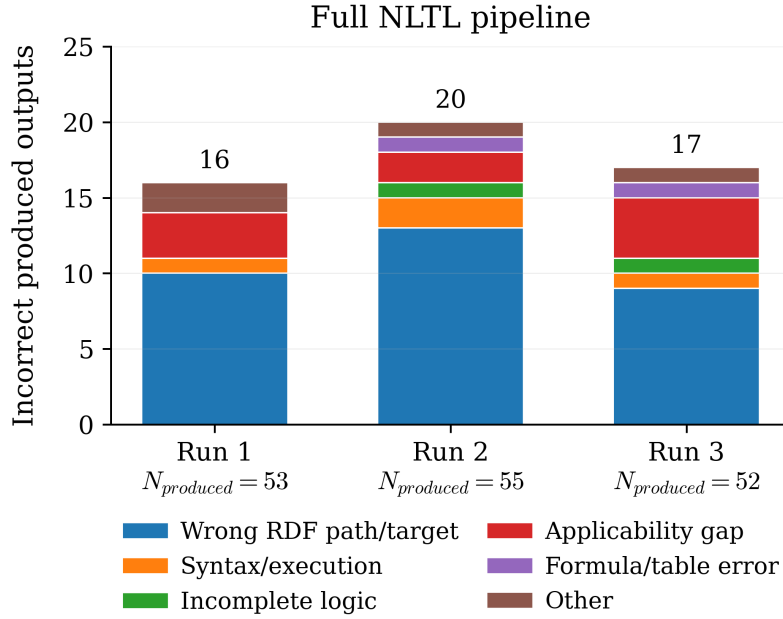


Figure 5: Failure-mode distribution among incorrect produced outputs for the full NLTL pipeline across the three repeated runs.

For the full NLTL pipeline, the number of incorrect produced outputs was lower than in the two ablation configurations in all three runs. However, the remaining errors show that validator-guided repair was not sufficient to eliminate semantic mistakes. Some accepted artifacts were still wrong because they checked only part of the requirement, used an imperfect applicability condition, selected an incorrect graph target, or produced the expected validation outcome for the wrong reason. This confirms that syntax validity, successful execution, and validator acceptance are useful checks, but they are not enough to guarantee regulatory correctness.

For NLTL without validator, the failure profile was more distributed. Wrong RDF path or target errors were lower than in the no-context configuration, while syntax or execution problems and incomplete validation logic formed a larger share of the remaining incorrect outputs. This indicates that ship-graph context reduced many basic vocabulary-alignment errors, but it did not fully solve the semantic translation problem. Even when the generated rule had access to the target graph context, it could still encode only part of the requirement, mishandle execution, or miss conditional logic.

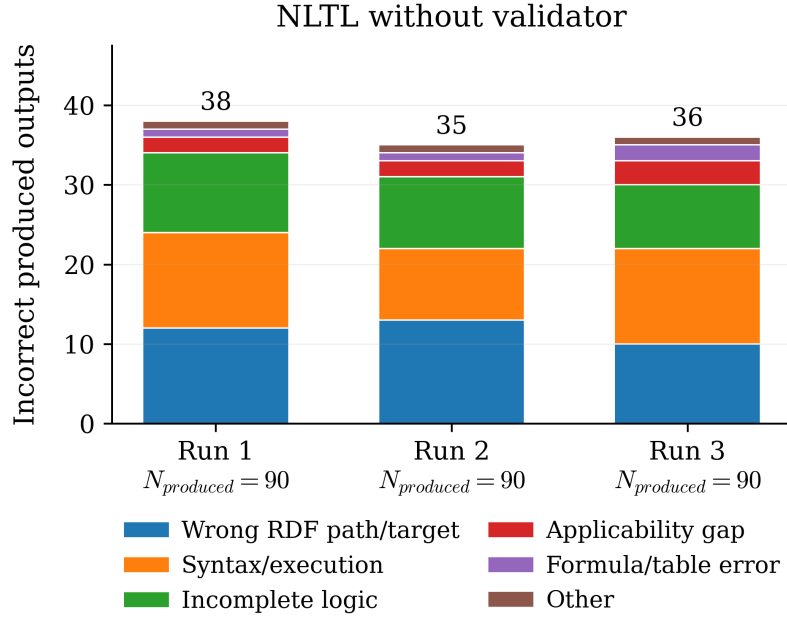


Figure 6: Failure-mode distribution among incorrect produced outputs for NLTL without validator across the three repeated runs.

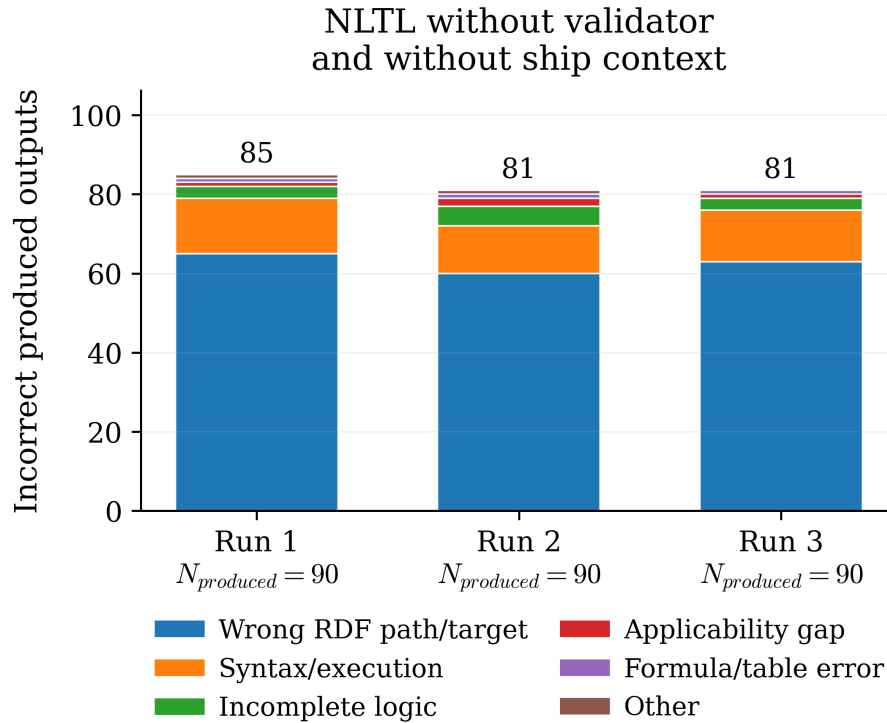


Figure 7: Failure-mode distribution among incorrect produced outputs for NLTL without validator and without ship context across the three repeated runs.

In the configuration without validator and without ship context, wrong RDF path or target errors dominated all three runs. This supports the ablation result that, without explicit ship-graph grounding, the model often selected plausible but incorrect RDF identifiers, target nodes, or value-access paths. The repeated-run pattern also shows that this was not an isolated Run 1 effect, but a stable failure mode of the no-context configuration.

Overall, the failure-mode distributions show that the dominant error mechanism depended strongly on the configuration. Ship-graph context mainly reduced vocabulary-alignment and target-selection errors, while validator-guided repair mainly reduced the number of accepted incorrect outputs without eliminating all semantic mistakes. These observations are discussed further in Section 5.1, where the ablation and failure-mode results are interpreted in relation to ship-graph grounding, validator-guided repair, and semantic correctness.

5 Discussion

This section interprets the NLTL evaluation in relation to the research questions, earlier research, uncertainty sources, and possible future use. The main result is that ship-graph grounding was the strongest factor affecting generation quality. Validator-guided repair did not increase the total number of correct outputs compared with NLTL without validator, which kept ship-graph context but removed validator-guided repair. However, it made the full pipeline more selective and occasionally improved the specificity of individual rules. The results therefore indicate that NLTL can assist SHACL rule authoring for selected maritime regulatory requirements, but the generated rules must still be treated as reviewable engineering artifacts rather than authoritative compliance decisions.

5.1 Analysis of the results

The clearest result is the effect of ship-graph context. Across the three repeated runs, NLTL without validator achieved $59.6\% \pm 1.7$ pp overall accuracy, while NLTL without validator and without ship context achieved only $8.5\% \pm 2.6$ pp. Because both configurations produced an output for every case, this difference mainly reflects semantic correctness rather than output coverage. This supports the interpretation that the model could not reliably infer the implemented RDF vocabulary, target nodes, or value-access paths from the regulation text and few-shot examples alone.

The comparison between the full NLTL pipeline and NLTL without validator shows a coverage–precision tradeoff. The full pipeline produced fewer accepted outputs, which reduced its overall accuracy to $39.6\% \pm 1.3$ pp, but its accuracy among produced outputs was higher at $66.9\% \pm 3.1$ pp. In contrast, NLTL without validator produced outputs for all cases and achieved $59.6\% \pm 1.7$ pp for both accuracy among produced outputs and overall accuracy. This suggests that the validator loop mainly acted as a selective filtering and repair mechanism rather than as a mechanism that maximized the total number of correct outputs.

Supplementary Welch’s t-tests were consistent with these observed differences and indicated statistically significant differences for the selected pairwise comparisons. These tests support the interpretation that ship context had a strong effect and that the full pipeline differed from NLTL without validator in both coverage and precision. However, the tests should be interpreted together with the small number of repeated runs. With $n = 3$, the sample standard deviations are themselves uncertain estimates of run-to-run variability, and additional runs could provide a clearer picture of the stability and strength of the observed differences.

This distinction is important for interpreting the role of validator-guided repair. The validator loop should not be described as making the full system better in every sense. In this evaluation, its main measurable effect was selectivity rather than higher total correctness. It filtered or aborted many cases that did not satisfy the acceptance criteria, and in some individual cases it helped produce a more specific rule. However, it did not outperform NLTL without validator in total number of correct outputs. This suggests that validator-guided repair is useful, but its benefit depends on the quality of

the validator judgment, the clarity of repair feedback, and the generator’s ability to use that feedback without introducing new errors.

The results also answer the research questions at the discussion level. For RQ1, the experiments show that LLMs can generate executable SHACL constraints from selected maritime regulatory cases when the input is structured and grounded in the target ship graph. For RQ2, the results show that generated SHACL artifacts must be checked through multiple layers: block extraction, Turtle syntax parsing, pySHACL execution, graph-alignment assessment, and manual semantic correctness review. For RQ3, the ablation study shows that ship-graph grounding had the strongest effect on correctness, while validator-guided repair mainly changed the selectivity and precision of accepted outputs rather than maximizing the total number of correct outputs.

The requirement category also affected the observed performance. The pipeline was strongest when the requirement could be represented as a direct threshold check, enumeration check, stored-value comparison, or readiness check over a variable already present in the ship graph. Static requirements were therefore more suitable for SHACL generation when the target value and comparison condition were clear. Static calculation cases were more reliable when the generated rule compared an existing or precomputed graph value rather than reconstructing the full regulatory formula inside SHACL-SPARQL. Formula-heavy cases were more fragile because they required the model to preserve mathematical meaning, lookup-table logic, and regulatory applicability conditions.

The complex readiness and dynamic readiness categories require careful interpretation. In many of these cases, a correct generated rule did not prove full regulatory compliance. Instead, it checked whether the ship graph contained the variable, output slot, or analysis evidence required for later compliance verification. This is still useful in a collaborative engineering workflow because missing variables and missing analysis outputs can block downstream validation. However, a readiness rule should not be interpreted as proof that the underlying engineering calculation, simulation, or fatigue assessment has been performed correctly.

Appendix C provides representative examples of correctly converted regulatory inputs from the full NLTL pipeline. These examples include a direct static value check, a static calculation ratio check, a complex readiness check, and a dynamic readiness check. They illustrate that correct generated outputs were not limited to one isolated requirement type. In the static nominal ice-pressure case, the generated rule directly checks the required stored value. In the blade-stress acceptability case, the generated rule computes a ratio from two graph values and compares it with the regulatory threshold. In the hull-geometry readiness case, the generated rule checks whether all required geometry variables are present in the ship graph. In the dynamic shaft-response case, the generated rule checks for condition-specific evidence slots needed for later time-domain vibration assessment.

These examples also clarify the boundary of what a correct generated SHACL artifact means in this thesis. For direct static and simple static calculation cases, a correct rule can represent a direct compliance check against the available graph values. For complex and dynamic cases, however, a correct rule is often a readiness-style rule: it verifies that required variables, outputs, or analysis evidence are represented in the

graph, but it does not verify the physical correctness of the underlying calculation, simulation, or fatigue assessment. The examples should therefore be read as illustrative accepted rule packages, not as proof that the system can automatically approve regulatory compliance without expert review.

Overall, NLTL is best understood as a grounded rule-authoring support pipeline. It can produce executable SHACL drafts for expert review for a meaningful subset of selected TRAFICOM-derived cases, especially when the relevant variables and graph paths are available. However, the results also show that generated rules remain sensitive to vocabulary coverage, graph modeling choices, applicability conditions, formula interpretation, and LLM non-determinism. The pipeline can support expert work, but it cannot replace expert rule authoring or regulatory interpretation.

5.2 Comparison to earlier research

The results are consistent with earlier work on semantic-web-based compliance checking, but they also highlight the additional difficulty introduced by automatic rule generation. Previous work has already shown that RDF and SHACL are suitable technologies for representing structured data and validating it against explicit constraints. This thesis extends that direction by evaluating the rule-authoring step: how LLM-assisted generation can help convert selected maritime regulatory requirements into executable SHACL rules that must align with a concrete RDF ship graph.

The work also extends earlier ontology-enabled engineering validation and variable-standardisation research. ReqTLo showed how formal requirement models can support simulation-linked validation and SHACL-based consistency checking [9]. AnchorMap showed that heterogeneous engineering variable labels can be aligned to a canonical JSON-LD vocabulary using selective LLM-assisted automation with human review for uncertain mappings [7]. The present thesis addresses the authoring step that remains after such structured identifiers and validation infrastructure exist: translating selected natural-language regulatory requirements into executable constraints that use the correct graph vocabulary and access paths.

The ablation results agree with ontology-guided and knowledge-graph-guided LLM studies. Mustafa et al. show that ontology context can support ODRL policy generation from natural language [16], and Qualis argues that knowledge-graph grounding can reduce hallucinated model elements in SysML v2 generation [55]. The present results provide similar evidence in the maritime SHACL setting. Without ship-graph context, the generator produced almost no correct rules. With ship-graph context, correctness increased substantially. This supports the broader conclusion that LLMs should not generate formal engineering artifacts from natural-language requirements alone; they need access to the target schema, vocabulary, or graph context.

The results also nuance the comparison with verification-guided generation research. Han et al.’s PiVe framework uses verifier feedback to improve graph-based generation [59], and AgentODRL separates generation, assessment, and repair for policy generation [54]. NLTL follows the same general principle by combining

generation with deterministic checks and validator-guided feedback. However, the ablation results show that feedback alone was not the dominant source of correctness in this case. The strongest improvement came from ship-graph context, because many incorrect outputs were caused by graph-alignment errors rather than only syntax or formatting errors. Validator-guided repair was still useful because it made the developed pipeline more selective and, in some accepted outputs, helped preserve more of the requirement-specific structure of the source text. However, it also reduced output coverage and could not guarantee semantic correctness. This suggests that verifier-in-the-loop systems for engineering rule generation should be evaluated by final artifact correctness, coverage, and failure modes, not only by whether a repair loop is present.

Compared with template-based natural-language-to-SHACL approaches, NLTL studies a less controlled generation setting. Donkers and Petrova convert fire safety regulations into SHACL shapes using linguistic analysis and predefined templates, with strongest results for simpler and more regular requirements [13]. NLTL instead uses an LLM to generate SHACL-SPARQL from structured cases, few-shot examples, and ship-graph context. This gives the method more flexibility, but also creates additional failure modes such as invented variables, wrong RDF paths, incomplete logic, and weak applicability handling. The comparison suggests that LLM-based generation can handle more varied inputs, but it requires stronger grounding, execution checks, and expert review.

The findings also connect to broader natural-language-to-logic research. Yang et al. show that LLMs can translate natural language into formal logical representations, while also identifying errors in predicate selection, condition handling, and semantic alignment [48]. Similar issues appeared in this thesis, but in RDF/SHACL form. The model often captured the general regulatory idea but encoded it using the wrong RDF identifier, target, or value-access path. This is especially important for SHACL generation because a plausible term is not enough: the rule must use the exact identifiers implemented in the target graph.

5.3 Uncertainty and error analysis

The results should be interpreted with several uncertainty sources in mind. The first is manual correctness assessment. Manual evaluation was necessary because syntax validity and successful pySHACL execution do not prove regulatory correctness. A generated rule can execute successfully while checking the wrong node, using the wrong path, omitting an applicability condition, or passing because it does not bind to the intended data. The final correctness judgment therefore depended on the relationship between the source requirement, the ship graph, the generated SHACL, and the observed validation behaviour.

The second uncertainty source is the use of one main RDF ship graph as the validation target. This made the experiment controlled and traceable, but it limited the ability to test whether generated rules generalize across ship variants. A rule that behaves correctly on one graph may fail on another graph with a different component structure, missing variable, alternative propulsion arrangement, or different

applicability condition. This limitation is especially relevant for propeller, vibration, fatigue, and other analysis-dependent requirements.

The third uncertainty source is the interpretation of readiness checks. Many complex and dynamic cases were evaluated as data-readiness checks rather than full compliance checks. A correct readiness rule can verify that a required output or analysis slot exists in the graph, but it does not verify that the simulation was performed correctly, that numerical values are physically valid, or that all units and assumptions are consistent. This limitation does not invalidate the results, but it limits what a correct output means for complex and dynamic cases.

The fourth uncertainty source is run-to-run variability. Each configuration was repeated three times, which was sufficient to show that the aggregate metrics were not identical across runs, but $n = 3$ is still a small sample for estimating variability. The reported standard deviations should therefore be interpreted as descriptive summaries of the three observed runs rather than stable estimates of the true variability of the pipeline. This limitation also affects the supplementary Welch's t -tests, because the variance estimates used by the tests are based on only three runs per configuration. Increasing the number of runs would have improved the uncertainty estimates, but it was practically difficult within the scope of this thesis because every produced SHACL artifact required manual semantic assessment. Across the reported experiments, the manual review covered 700 produced artifacts: 160 from the full NLTL pipeline, 270 from NLTL without validator, and 270 from NLTL without validator and without ship context. Each artifact had to be checked against the source requirement, generated SHACL, ship-graph alignment, execution behaviour, and correctness of the validation reason. Additional repeated runs would therefore have substantially increased the manual evaluation workload.

The validator itself is another source of uncertainty because it is also an LLM-based component. Although the validator receives deterministic evidence from RDFLib and pySHACL, its semantic judgment can still produce false acceptance or false rejection. The case-by-case evaluation design also limits the conclusions that can be drawn about complete regulation-wide rule libraries, because the pipeline does not maintain persistent memory across regulatory cases or automatically resolve dependencies between clauses.

The failure-mode analysis helps identify where errors entered the pipeline across repeated runs. Without ship context, wrong RDF path or target errors dominated all three runs, confirming that many incorrect outputs came from unsupported or incorrectly selected graph identifiers. With ship context, these basic alignment errors were reduced, but other errors remained, including syntax or execution problems, incomplete validation logic, formula or table mistakes, and applicability gaps. This pattern suggests that graph context solves many vocabulary-alignment problems, but not the full semantic translation problem. Once the model has access to plausible identifiers, the harder task is selecting the correct variables, preserving applicability logic, and deciding whether the rule should check a stored value, a derived formula, or a readiness condition.

A stronger future evaluation would therefore require counterexample-style testing. Instead of executing each generated rule only against the original case-study graph,

the rule should also be tested against modified graphs where relevant variables are removed, thresholds are changed, or applicability conditions are altered. If the generated rule does not report a violation in the modified graph, it is not checking the intended condition. Such tests would provide stronger evidence that the rule fires for the correct reason.

5.4 Scientific impact

The scientific value of this thesis is that it evaluates LLM-generated regulatory rules as executable artifacts rather than as text alone. The work connects natural-language maritime regulation, RDF ship design data, SHACL execution, validator-guided repair, and manual semantic assessment in one experimental workflow. This is important because formal artifact generation cannot be evaluated only by surface plausibility; the artifact must run against a data model and represent the intended requirement.

The novelty of the work is also in the specific combination of problem setting, artifact, and evaluation. Donkers and Petrova studied template-based conversion of fire-safety regulations into SHACL shapes, whereas this thesis evaluates LLM-assisted SHACL generation for maritime regulatory clauses against a concrete RDF ship graph [13]. Westermann et al. studied LLM-generated SHACL constraints for AutomationML models, whereas this thesis focuses on TRAFICOM-derived maritime requirements and ship-design graph grounding [15]. ReqTLo and AnchorMap provide important upstream foundations for requirement representation and variable standardisation, but they do not evaluate automatic generation of regulatory SHACL rules from natural-language maritime clauses [7, 9]. The contribution of this thesis is therefore not only that SHACL rules are executed, but that rule generation, graph grounding, execution evidence, validator-guided repair, and manual semantic assessment are evaluated together in one maritime engineering workflow.

A second contribution is the separation of ship-graph grounding from validator-guided repair. The ablation study shows that target-model grounding was the dominant factor in correctness. This provides empirical support for a broader claim in LLM-assisted engineering automation: formal outputs should be generated with explicit access to the vocabulary, schema, or graph they are expected to operate on. Without this grounding, the model may produce text that appears technically plausible but is not executable or meaningful for the actual engineering data.

A third contribution is the evaluation framing based on output rate, accuracy among produced outputs, overall accuracy, and repeated-run mean and sample standard deviation. These metrics are especially important for verifier-in-the-loop systems because such systems may abort uncertain cases. Reporting only accuracy among produced outputs would hide aborted cases, while reporting only overall accuracy would hide the higher correctness rate among accepted outputs. Reporting repeated-run summary statistics adds a descriptive view of run-to-run variability. The combination of metrics gives a more honest view of both coverage and selectivity.

A fourth contribution is the distinction between direct compliance checks and readiness checks. In engineering domains, many regulatory requirements depend on calculations, simulations, or external evidence that SHACL cannot produce by

itself. The thesis shows that SHACL can still be useful for checking whether required analysis artifacts are represented in the graph. However, it also shows that such checks must not be confused with full physical or regulatory verification.

5.5 Practical impact

In practical terms, NLTL can support engineers by producing SHACL drafts for expert review and by linking those drafts to syntax checks, execution results, validator feedback, and manual evaluation. This can reduce part of the effort involved in moving from regulatory prose to executable validation rules. It can also help expose missing variables, inconsistent graph modeling, or unclear applicability conditions earlier in a design workflow.

The approach is most useful for early-stage compliance support and data-completeness checking. In collaborative ship design, relevant information may come from shipyards, equipment manufacturers, simulation tools, and regulatory documents. A grounded SHACL-generation pipeline can help connect these sources by drafting rules that refer to the same RDF vocabulary as the ship graph. This supports traceability from a source requirement to a generated validation artifact and then to a validation report.

In a longer-term engineering workflow, this type of pipeline could support the gradual creation of reviewable regulation-to-rule libraries. Instead of manually writing every SHACL constraint from the beginning, engineers could use NLTL-style generation to draft candidate rules, execute them against ship-design graphs, inspect the validation evidence, and then approve, revise, or reject the generated artifacts. Such a workflow would fit naturally with graph-based ship design, digital-twin-enabled collaboration, and shared engineering vocabularies, where regulatory clauses, design variables, simulation outputs, and validation reports can be linked through common identifiers. The immediate practical value is therefore not autonomous approval, but earlier detection of missing data, unclear graph modeling, and potential non-compliance during design development.

However, the system should not be used as an autonomous approval mechanism. The main practical risk is not simply invalid syntax, because syntax errors are relatively easy to detect. The more serious risk is a plausible-looking rule that executes successfully while checking the wrong graph element or only part of the intended requirement. For this reason, generated rules should remain subject to expert review, especially for formula-heavy, dynamic, or applicability-dependent requirements.

The results also show that the quality of the underlying ship graph is central to practical deployment. The generator can only produce correct SHACL if the relevant components, variables, and value-access paths are available and consistently represented. Industrial use would therefore require a maintained vocabulary, clear variable naming conventions, and preferably direct links between regulation terms and graph identifiers. A stronger test environment would also be needed, including conforming and non-conforming graph variants for each rule before it is added to a production validation library.

5.6 Future research and open research questions

The first future research direction is stronger ontology and schema grounding. The ablation results show that ship context is essential, but the remaining failures show that context should be more structured and verifiable. Future versions should connect the generator directly to the JSON-LD schema, ontology, or controlled variable dictionary. Although AnchorMap was not part of the implemented NLTL pipeline, it provides one possible upstream mechanism for maintaining canonical engineering identifiers. Retrieval-augmented generation could then provide the exact identifiers, definitions, and value-access paths relevant to each regulatory case [7, 53].

The second direction is deterministic target and path verification. Before a generated rule is accepted, the pipeline should check whether its target nodes bind to actual graph nodes and whether the paths used in SHACL or SPARQL occur in the ship graph. This would catch many rules that are syntactically valid but ineffective because they use unsupported classes or properties. Such checks should be performed before asking the validator model to assess semantic correctness.

The third direction is counterexample-based testing. Each generated rule should be tested against graph variants that are expected to conform and variants that are expected to violate the rule. For example, if a readiness rule requires `ssp:PeakPropellerTorque`, a modified graph should remove that variable and the rule should report a violation. This follows the broader idea of using counterexamples or solver feedback to expose incorrect generated formal artifacts [52]. Such tests would provide stronger evidence that the generated rule checks the intended condition rather than merely producing the expected result on one graph.

The fourth direction is clearer classification of generated rule types. Future systems should distinguish direct compliance checks, stored-value checks, formula-input completeness checks, and simulation-output readiness checks. This would reduce overclaiming and help engineers understand what kind of evidence each generated rule provides.

The fifth direction is persistent learning across cases. The current pipeline passes feedback within a single case but does not reuse successful repairs across later cases. Many observed errors are recurring, such as wrong target classes, wrong value-access paths, and missing applicability logic. A memory or retrieval mechanism could store successful corrections and make them available for similar future cases [58].

The sixth direction is explicit abstention and confidence calibration. The full NLTL pipeline already behaves selectively because it can abort cases that do not satisfy the acceptance criteria. However, the validator is not yet calibrated as a reliable abstention mechanism. Work on selective generation shows that model self-evaluation can help decide when generation should be withheld [65], while recent surveys describe abstention as an important mechanism for reducing hallucination and improving safety in LLM systems [66]. Future NLTL versions could therefore distinguish more clearly between accepted rules, repairable rules, and cases where the system should explicitly refuse to produce an executable rule without additional human or data-model input.

The seventh direction is broader evaluation. The current benchmark uses selected TRAFICOM ice-class requirements and one main ship graph. Future work should

test additional ship graphs, vessel types, and regulatory sources. This would show whether the approach generalizes beyond the current case study and whether generated rules remain correct when component structures, variable availability, or applicability conditions change.

Finally, future research should investigate hybrid rule-authoring architectures. Template-based generation is more controlled for repeated threshold and required-variable patterns, while LLM-based generation is more flexible for varied natural-language cases [13, 15]. A practical system could use templates for common rule types and reserve LLM generation for cases where the structure is more complex. The results of this thesis suggest that such a hybrid architecture, combined with strong ontology grounding and deterministic verification, is a promising direction for reliable regulatory rule authoring.

6 Conclusions

This thesis developed and evaluated NLTL, an LLM-assisted pipeline for translating selected natural-language maritime regulatory requirements into executable SHACL constraints. The work addressed the gap between regulatory prose and machine-executable compliance checking by combining structured regulation cases, ship-graph grounding, deterministic syntax and execution checks, pySHACL execution, validator-guided repair, and manual semantic assessment. The pipeline was evaluated on 90 selected TRAFICOM ice-class regulatory cases against an RDF ship design graph.

The results answer the three research questions of the thesis. For RQ1, concerning the feasibility of generating executable SHACL constraints from selected natural-language maritime requirements, the results show that LLMs can produce usable rule artifacts when generation is structured around regulatory case files, few-shot examples, and explicit ship-graph context. The generated artifacts were not treated as text alone, but as executable SHACL constraints serialized in Turtle that had to be extracted, parsed, executed, and assessed against the intended regulatory meaning.

For RQ2, concerning how LLM-generated SHACL artifacts can be checked before acceptance, the thesis shows that a layered validation procedure is necessary. SHACL block extraction identifies whether the model produced a usable rule artifact, RDFLib checks Turtle syntax, pySHACL tests whether the generated shapes execute against the ship graph, and the validator model assesses the candidate using the original case, syntax evidence, execution evidence, and ship-graph context. Manual semantic review remains necessary because syntactic validity and successful execution do not guarantee that the rule checks the intended requirement.

For RQ3, concerning the effect of ship-graph grounding and validator-guided repair, the ablation results show that ship-graph grounding was the dominant factor in correctness. Across the three repeated runs, NLTL without validator achieved $59.6\% \pm 1.7$ pp overall accuracy, while NLTL without validator and without ship context achieved $8.5\% \pm 2.6$ pp. This indicates that the generator could not reliably infer the implemented RDF vocabulary, target nodes, and value-access paths from the regulation text and few-shot examples alone. Validator-guided repair did not increase the total number of correct outputs compared with NLTL without validator. Instead, its main measurable effect was selectivity: the full NLTL pipeline accepted fewer outputs, which reduced its overall accuracy, but the accepted outputs had a higher correctness rate. The validator loop should therefore be interpreted as a filtering and repair mechanism that can improve the precision of accepted artifacts, not as a guarantee of higher total correctness.

The main conclusion is that LLM-assisted SHACL generation can support regulatory rule authoring, but it cannot yet replace expert rule authoring or regulatory interpretation. NLTL produced correct executable SHACL artifacts for a meaningful subset of the selected cases, but the results also showed limitations in formula-heavy requirements, applicability conditions, graph vocabulary coverage, validator reliability, and run-to-run stability. A generated rule can be valid Turtle and executable SHACL while still targeting the wrong node, using an incorrect value-access path, omitting an applicability condition, or checking only part of the intended requirement.

NLTL should therefore be understood as a support tool for producing executable SHACL drafts for expert review. Its value is strongest in early-stage compliance and data-readiness workflows, where it can help engineers identify missing variables, draft validation rules, and trace generated constraints back to source requirements. For safety-critical or regulatory approval contexts, expert review, counterexample testing, and validation against multiple ship-graph variants remain necessary.

Overall, the thesis demonstrates a feasible but limited path from natural-language maritime regulation to executable graph-based validation artifacts. The results support the broader conclusion that LLM-based formal artifact generation should be grounded in the target data model, checked with deterministic tools, and reviewed as an engineering artifact before use in compliance workflows.

References

- [1] R. Ala-Laurinaho, J. Autiosalo, S. Laine, U. Hakonen, and R. Viitala. “Paradigm Shift in Mechanical System Design: Toward Automated and Collaborative Design with Digital Twin Web”. In: *Software and Systems Modeling* 24.5 (2025), pp. 1475–1494. DOI: [10.1007/s10270-024-01215-8](https://doi.org/10.1007/s10270-024-01215-8).
- [2] T. Blochwitz, M. Otter, J. Åkesson, M. Arnold, C. Clauss, H. Elmqvist, M. Friedrich, A. Junghanns, J. Mauss, D. Neumerkel, H. Olsson, and A. Viel. “Functional Mockup Interface 2.0: The Standard for Tool Independent Exchange of Simulation Models”. In: *Proceedings of the 9th International Modelica Conference*. Linköping University Electronic Press, 2012, pp. 173–184. DOI: [10.3384/ecp12076173](https://doi.org/10.3384/ecp12076173).
- [3] R. Hällqvist, R. C. Munjulury, R. Braun, M. Eek, and P. Krus. “Engineering Domain Interoperability Using the System Structure and Parameterization (SSP) Standard”. In: *Proceedings of the 14th International Modelica Conference*. Linköping University Electronic Press, 2021. DOI: [10.3384/ecp2118137](https://doi.org/10.3384/ecp2118137).
- [4] ISO. *ISO 19848:2024 Ships and Marine Technology – Standard Data for Shipboard Machinery and Equipment*. Geneva, Switzerland, 2024. URL: <https://www.iso.org/standard/78262.html> (accessed on June 7, 2026).
- [5] Z. Sivic, H. Al-Shami, R. Ala-Laurinaho, and R. Viitala. “Automated Transformation of SysML v2 Conceptual Models into System Simulation Structures for Maritime Co-Simulation”. In: *Proceedings of the 40th ECMS International Conference on Modelling and Simulation*. Vol. 40. Communications of the ECMS 1. Grimstad, Norway: European Council for Modelling and Simulation, 2026, pp. 818–826. DOI: [10.7148/2026-0818](https://doi.org/10.7148/2026-0818).
- [6] U. Muhammad. “Automating Classification in Maritime Industry Using Multimodal Retrieval-Augmented Generation”. Master’s thesis. Espoo, Finland: Aalto University, July 31, 2025. URL: <https://urn.fi/URN:NBN:fi:aalto-202508186258> (accessed on June 27, 2026).
- [7] S. A. Rana, H. Al-Shami, R. Ala-Laurinaho, C. Yang, and R. Viitala. “AnchorMap: A Multi-Agent Pipeline for Variable Standardisation in Maritime Engineering”. In: *Proceedings of the 40th ECMS International Conference on Modelling and Simulation*. Vol. 40. Communications of the ECMS 1. Grimstad, Norway: European Council for Modelling and Simulation, 2026, pp. 389–396. DOI: [10.7148/2026-0389](https://doi.org/10.7148/2026-0389).
- [8] Finnish Transport and Communications Agency Traficom. *Ice Class Regulations and the Application Thereof*. Register number: TRAFICOM/68863/03.04.01.00/2021. Entry into force: 2021-07-05. Published in Finlex, Collections of regulations. July 1, 2021. URL: <https://www.finlex.fi/en/authorities/regulations/traficom-transport-maritime/2021/47238> (accessed on May 18, 2026).

- [9] H. Al-Shami, R. Ala-Laurinaho, M. Eklund, and R. Viitala. *ReqTLo: Requirements Temporal Logic Ontology for Simulation-based Validation of Systems in Digital Twin-enabled Collaborative Design*. TechRxiv preprint. Apr. 2025. DOI: [10.36227/techrxiv.174537545.54263454](https://doi.org/10.36227/techrxiv.174537545.54263454).
- [10] C. Eastman, J.-K. Lee, Y.-S. Jeong, and J.-K. Lee. “Automatic Rule-Based Checking of Building Designs”. In: *Automation in Construction* 18.8 (2009), pp. 1011–1033. DOI: [10.1016/j.autcon.2009.07.002](https://doi.org/10.1016/j.autcon.2009.07.002).
- [11] J. Dimyadi and R. Amor. “Automated Building Code Compliance Checking: Where Is It At?” In: *Proceedings of the 19th CIB World Building Congress*. 2013, pp. 172–185. URL: https://www.researchgate.net/publication/236681238_Automated_Building_Code_Compliance_Checking_-_Where_is_it_at (accessed on June 7, 2026).
- [12] J. Zhang and N. M. El-Gohary. “Semantic NLP-Based Information Extraction from Construction Regulatory Documents for Automated Compliance Checking”. In: *Journal of Computing in Civil Engineering* 30.2 (2016), p. 04015014. DOI: [10.1061/\(ASCE\)CP.1943-5487.0000346](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000346).
- [13] A. J. A. Donkers and E. Petrova. “Converting Fire Safety Regulations to SHACL Shapes Using Natural Language Processing”. In: *Proceedings of the 3rd International Workshop on Natural Language Processing for Knowledge Graph Creation (NLP4KGC 2024)*. Vol. 3874. CEUR Workshop Proceedings. CEUR-WS.org, 2024, pp. 97–111. URL: <https://ceur-ws.org/Vol-3874/paper7.pdf> (accessed on June 7, 2026).
- [14] A. Cimmino, A. Fernández-Izquierdo, and R. García-Castro. “Astrea: Automatic Generation of SHACL Shapes from Ontologies”. In: *The Semantic Web*. Vol. 12123. Lecture Notes in Computer Science. Springer, 2020, pp. 497–513. DOI: [10.1007/978-3-030-49461-2_29](https://doi.org/10.1007/978-3-030-49461-2_29).
- [15] T. Westermann, A. Köcher, and F. Gehlhoff. *Automated Validation of Textual Constraints Against AutomationML via LLMs and SHACL*. arXiv preprint. June 2025. arXiv: [2506.10678](https://arxiv.org/abs/2506.10678) [cs.AI].
- [16] D. M. Mustafa, A. Nadgeri, D. Collarana, B. T. Arnold, C. Quix, C. Lange, and S. Decker. *From Instructions to ODRL Usage Policies: An Ontology Guided Approach*. arXiv preprint of the LLM+KG Workshop at VLDB 2024 paper. 2025. DOI: [10.48550/arXiv.2506.03301](https://doi.org/10.48550/arXiv.2506.03301).
- [17] N. Bus, A. Roxin, G. Picinbono, and M. Fahad. *Towards French Smart Building Code: Compliance Checking Based on Semantic Rules*. arXiv preprint. 2019. arXiv: [1910.00334](https://arxiv.org/abs/1910.00334) [cs.AI].
- [18] F. Noardo, T. Wu, K. Arroyo Otori, T. Krijnen, and J. Stoter. “IFC Models for Semi-Automating Common Planning Checks for Building Permits”. In: *Automation in Construction* 134 (2022), p. 104097. DOI: [10.1016/j.autcon.2021.104097](https://doi.org/10.1016/j.autcon.2021.104097).

- [19] M. J. Bommarito, D. M. Katz, and E. M. Detterman. *LexNLP: Natural Language Processing and Information Extraction for Legal and Regulatory Texts*. arXiv preprint. 2018. arXiv: [1806.03688](https://arxiv.org/abs/1806.03688) [cs.CL].
- [20] A. Sleimi, N. Sannier, M. Sabetzadeh, L. Briand, M. Ceci, and J. Dann. “An Automated Framework for the Extraction of Semantic Legal Metadata from Legal Texts”. In: *Empirical Software Engineering* 26.43 (2021). DOI: [10.1007/s10664-020-09933-5](https://doi.org/10.1007/s10664-020-09933-5).
- [21] H. Hettiarachchi, A. Dridi, M. M. Gaber, P. Parsafard, N. Bocaneala, K. Breitenfelder, G. Costa, M. Hedblom, M. Juganaru-Mathieu, T. Mecharnia, S. Park, H. Tan, A.-R. H. Tawil, and E. Vakaj. “CODE-ACCORD: A Corpus of Building Regulatory Data for Rule Generation towards Automatic Compliance Checking”. In: *Scientific Data* 12.170 (2025). DOI: [10.1038/s41597-024-04320-x](https://doi.org/10.1038/s41597-024-04320-x).
- [22] OASIS. *LegalRuleML Core Specification Version 1.0*. OASIS Standard. Available online: [OASIS LegalRuleML Core Specification Version 1.0](https://www.oasis-open.org/standards/LegalRuleML-Core-Specification-Version-1.0/). Accessed 7 June 2026. OASIS Open, Aug. 30, 2021.
- [23] J. Arias, S. Törmä, M. Carro, and G. Gupta. “Building Information Modeling Using Constraint Logic Programming”. In: *Theory and Practice of Logic Programming* 22.5 (2022), pp. 723–738. DOI: [10.1017/S1471068422000138](https://doi.org/10.1017/S1471068422000138).
- [24] B. Nuseibeh and S. Easterbrook. “Requirements Engineering: A Roadmap”. In: *Proceedings of the Conference on the Future of Software Engineering*. ICSE '00. Association for Computing Machinery, 2000, pp. 35–46. DOI: [10.1145/336512.336523](https://doi.org/10.1145/336512.336523).
- [25] P. Zave and M. Jackson. “Four Dark Corners of Requirements Engineering”. In: *ACM Transactions on Software Engineering and Methodology* 6.1 (1997), pp. 1–30. DOI: [10.1145/237432.237434](https://doi.org/10.1145/237432.237434).
- [26] ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*. Geneva, Switzerland, 2018. URL: <https://www.iso.org/standard/72089.html> (accessed on June 7, 2026).
- [27] H. Femmer, D. Méndez Fernández, S. Wagner, and S. Eder. “Rapid Quality Assurance with Requirements Smells”. In: *Journal of Systems and Software* 123 (2017), pp. 190–213. DOI: [10.1016/j.jss.2016.02.047](https://doi.org/10.1016/j.jss.2016.02.047).
- [28] M. B. Dwyer, G. S. Avrunin, and J. C. Corbett. “Patterns in Property Specifications for Finite-State Verification”. In: *Proceedings of the 21st International Conference on Software Engineering*. ICSE '99. Association for Computing Machinery, 1999, pp. 411–420. DOI: [10.1145/302405.302672](https://doi.org/10.1145/302405.302672).
- [29] S. Konrad and B. H. C. Cheng. “Real-Time Specification Patterns”. In: *Proceedings of the 27th International Conference on Software Engineering*. ICSE '05. Association for Computing Machinery, 2005, pp. 372–381. DOI: [10.1145/1062455.1062526](https://doi.org/10.1145/1062455.1062526).

- [30] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske. “A Property Specification Pattern Catalog for Real-Time System Verification with UPPAAL”. In: *Information and Software Technology* 154 (2023), p. 107100. DOI: [10.1016/j.infsof.2022.107100](https://doi.org/10.1016/j.infsof.2022.107100).
- [31] A. Cimatti, M. Roveri, A. Susi, and S. Tonetta. “Formalization and Validation of Safety-Critical Requirements”. In: *Proceedings of the 7th International Workshop on Formal Methods for Aerospace*. Vol. 20. Electronic Proceedings in Theoretical Computer Science. 2010, pp. 68–75. DOI: [10.4204/EPTCS.20.7](https://doi.org/10.4204/EPTCS.20.7).
- [32] W3C. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation. Feb. 25, 2014. URL: <https://www.w3.org/TR/rdf11-concepts/> (accessed on June 7, 2026).
- [33] W3C. *RDF 1.1 Turtle: Terse RDF Triple Language*. W3C Recommendation. Feb. 25, 2014. URL: <https://www.w3.org/TR/turtle/> (accessed on June 7, 2026).
- [34] W3C. *JSON-LD 1.1: A JSON-based Serialization for Linked Data*. W3C Recommendation. July 16, 2020. URL: <https://www.w3.org/TR/json-ld11/> (accessed on May 18, 2026).
- [35] W3C. *SPARQL 1.1 Query Language*. W3C Recommendation. Mar. 21, 2013. URL: <https://www.w3.org/TR/sparql11-query/> (accessed on June 7, 2026).
- [36] W3C. *OWL 2 Web Ontology Language Document Overview*. W3C Recommendation, Second Edition. Dec. 11, 2012. URL: <https://www.w3.org/TR/owl2-overview/> (accessed on June 7, 2026).
- [37] W3C. *Shapes Constraint Language (SHACL)*. W3C Recommendation. July 20, 2017. URL: <https://www.w3.org/TR/shacl/> (accessed on May 18, 2026).
- [38] B. Bogaerts, M. Jakubowski, and J. Van den Bussche. *SHACL: A Description Logic in Disguise*. arXiv preprint. 2021. arXiv: [2108.06096](https://arxiv.org/abs/2108.06096) [cs.LO].
- [39] P. Pareti and G. Konstantinidis. “A Review of SHACL: From Data Validation to Schema Reasoning for RDF Graphs”. In: *Reasoning Web. Declarative Artificial Intelligence*. Vol. 13100. Lecture Notes in Computer Science. Springer, 2021, pp. 115–144. DOI: [10.1007/978-3-030-95481-9_6](https://doi.org/10.1007/978-3-030-95481-9_6).
- [40] P. Pauwels, S. Zhang, and Y.-C. Lee. “Semantic Web Technologies in AEC Industry: A Literature Review”. In: *Automation in Construction* 73 (2017), pp. 145–165. DOI: [10.1016/j.autcon.2016.10.003](https://doi.org/10.1016/j.autcon.2016.10.003).
- [41] E. Nuyts, J. Werbrouck, R. Verstraeten, and L. Deprez. “Validation of Building Models against Legislation Using SHACL”. In: *Proceedings of the 11th Linked Data in Architecture and Construction Workshop (LDAC2023)*. Vol. 3633. CEUR Workshop Proceedings. CEUR-WS.org, 2023, pp. 164–175. URL: <https://ceur-ws.org/Vol-3633/paper13.pdf> (accessed on June 7, 2026).

- [42] L. Robaldo and K. J. Adebayo. *Compliance Checking in Reified IO Logic via SHACL*. arXiv preprint. 2021. arXiv: [2110.07033](https://arxiv.org/abs/2110.07033) [cs.LO].
- [43] RDFLib. *RDFLib: Python Library for Working with RDF*. Software documentation. 2026. URL: <https://rdflib.readthedocs.io/> (accessed on June 7, 2026).
- [44] RDFLib. *pySHACL: A Python validator for SHACL*. Software documentation and source repository. 2026. URL: <https://github.com/RDFLib/pySHACL> (accessed on May 18, 2026).
- [45] M. Cheatham and P. Hitzler. “String Similarity Metrics for Ontology Alignment”. In: *The Semantic Web – ISWC 2013*. Vol. 8219. Lecture Notes in Computer Science. Springer, 2013, pp. 294–309. DOI: [10.1007/978-3-642-41338-4_19](https://doi.org/10.1007/978-3-642-41338-4_19).
- [46] Y. He, J. Chen, H. Dong, and I. Horrocks. *Exploring Large Language Models for Ontology Alignment*. arXiv preprint. 2023. arXiv: [2309.07172](https://arxiv.org/abs/2309.07172) [cs.AI].
- [47] M. Rindarøy, H. Nordahl, S. Sadjina, S. Skjong, and M. Hagaseth. “Adding Higher-Level Semantics to Functional Mock-up Units for Easier, Faster, and More Robust Co-Simulation Connections”. In: *Software and Systems Modeling* 24 (2025), pp. 471–487. DOI: [10.1007/s10270-024-01244-3](https://doi.org/10.1007/s10270-024-01244-3).
- [48] Y. Yang, S. Xiong, A. Payani, E. Shareghi, and F. Fekri. “Harnessing the Power of Large Language Models for Natural Language to First-Order Logic Translation”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 6942–6959. DOI: [10.18653/v1/2024.acl-long.375](https://doi.org/10.18653/v1/2024.acl-long.375).
- [49] A. Randles, D. O’Sullivan, and C. Debruyne. “The R2[RML]-ChatGPT Framework: Generating RDF Sample Data, SPARQL Queries and SHACL Shapes from R2RML Mappings”. In: *Proceedings of the 5th International Workshop on Knowledge Graph Construction*. 2024. URL: <https://kg-construct.github.io/workshop/2024/resources/paper6.pdf> (accessed on June 7, 2026).
- [50] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12 (2023). DOI: [10.1145/3571730](https://doi.org/10.1145/3571730).
- [51] S. Lin, J. Hilton, and O. Evans. “TruthfulQA: Measuring How Models Mimic Human Falsehoods”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2022, pp. 3214–3252. DOI: [10.18653/v1/2022.acl-long.229](https://doi.org/10.18653/v1/2022.acl-long.229).

- [52] S. K. Jha, S. Jha, P. Lincoln, N. D. Bastian, A. Velasquez, R. Ewetz, and S. Neema. “Counterexample Guided Inductive Synthesis Using Large Language Models and Satisfiability Solving”. In: *MILCOM 2023 - 2023 IEEE Military Communications Conference*. IEEE, 2023, pp. 944–949. DOI: [10.1109/MILCOM58377.2023.10356332](https://doi.org/10.1109/MILCOM58377.2023.10356332).
- [53] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401> (accessed on June 7, 2026).
- [54] W. Zhong, K. Huang, and X. Du. “AgentODRL: A Large Language Model-based Multi-agent System for ODRL Generation”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 23. 2026, pp. 19477–19485. DOI: [10.1609/aaai.v40i23.39026](https://doi.org/10.1609/aaai.v40i23.39026).
- [55] R. A. Qualis. “Mitigating Hallucinations in SysML v2 Generation Using LLMs and a Tri-Layered Knowledge Graph Reasoning Framework”. In: *2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 2025, pp. 357–362. DOI: [10.1109/MODELS-C68889.2025.00054](https://doi.org/10.1109/MODELS-C68889.2025.00054).
- [56] L. M. Vieira da Silva, A. Köcher, F. Gehlhoff, and A. Fay. *On the Use of Large Language Models to Generate Capability Ontologies*. arXiv preprint. 2024. arXiv: [2404.17524](https://arxiv.org/abs/2404.17524) [cs.AI].
- [57] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. “Self-Refine: Iterative Refinement with Self-Feedback”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. URL: <https://arxiv.org/abs/2303.17651> (accessed on June 7, 2026).
- [58] N. Tandon, A. Madaan, P. Clark, and Y. Yang. “Learning to Repair: Repairing Model Output Errors after Deployment Using a Dynamic Memory of Feedback”. In: *Findings of the Association for Computational Linguistics: NAACL 2022*. Association for Computational Linguistics, 2022, pp. 339–352. DOI: [10.18653/v1/2022.findings-naacl.26](https://doi.org/10.18653/v1/2022.findings-naacl.26).
- [59] J. Han, N. Collier, W. Buntine, and E. Shareghi. “PiVe: Prompting with Iterative Verification Improving Graph-based Generative Capability of LLMs”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024, pp. 6702–6718. DOI: [10.18653/v1/2024.findings-acl.400](https://doi.org/10.18653/v1/2024.findings-acl.400).
- [60] A. R. Hevner, S. T. March, J. Park, and S. Ram. “Design Science in Information Systems Research”. In: *MIS Quarterly* 28.1 (2004), pp. 75–105. URL: <https://aisel.aisnet.org/misq/vol28/iss1/6/> (accessed on May 18, 2026).

- [61] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee. “A Design Science Research Methodology for Information Systems Research”. In: *Journal of Management Information Systems* 24.3 (2007), pp. 45–77. DOI: [10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302).
- [62] S. A. Rana. *Source Code for Translating Natural-Language Maritime Regulations into Machine-Readable Rules Using LLMs*. GitHub repository. 2026. URL: <https://github.com/saadahmedrana/Translating-Natural-Language-Maritime-Regulations-into-Machine-Readable-Rules-Using-LLMs> (accessed on June 18, 2026).
- [63] B. L. Welch. “The Generalisation of Student’s Problems When Several Different Population Variances Are Involved”. In: *Biometrika* 34.1–2 (1947), pp. 28–35. DOI: [10.1093/biomet/34.1-2.28](https://doi.org/10.1093/biomet/34.1-2.28).
- [64] G. D. Ruxton. “The Unequal Variance t-Test Is an Underused Alternative to Student’s t-Test and the Mann–Whitney U Test”. In: *Behavioral Ecology* 17.4 (2006), pp. 688–690. DOI: [10.1093/beheco/ark016](https://doi.org/10.1093/beheco/ark016).
- [65] J. Ren, Y. Zhao, T. Vu, P. J. Liu, and B. Lakshminarayanan. “Self-Evaluation Improves Selective Generation in Large Language Models”. In: *Proceedings on “I Can’t Believe It’s Not Better: Failure Modes in the Age of Foundation Models” at NeurIPS 2023 Workshops*. Vol. 239. Proceedings of Machine Learning Research. PMLR, 2023, pp. 49–64. URL: <https://proceedings.mlr.press/v239/ren23a.html> (accessed on June 7, 2026).
- [66] B. Wen, J. Yao, S. Feng, C. Xu, Y. Tsvetkov, B. Howe, and L. L. Wang. “Know Your Limits: A Survey of Abstention in Large Language Models”. In: *Transactions of the Association for Computational Linguistics* 13 (2025), pp. 529–556. DOI: [10.1162/tac1_a_00754](https://doi.org/10.1162/tac1_a_00754).

A Few-shot SHACL examples used for prompting

This appendix shows four compact few-shot examples used to guide the generator model. Each example contains the input requirement text and the expected SHACL-style output pattern. The examples were used as prompting guidance and should not be interpreted as complete or authoritative formalizations of the TRAFICOM regulation.

Static threshold check

Input requirement.

The open area of the strainer plates shall be no less than four times the inlet pipe sectional area.

Output pattern.

```
ice:StrainerPlateAreaShape a sh:NodeShape ;
  sh:targetNode ssp:iceShip ;
  sh:message "Strainer plate area ratio is below minimum." ;

  sh:sparql [
    sh:message "Strainer plate open area ratio ({?ratio}) is below the
required minimum of 4.0." ;
    sh:select """
      PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
      PREFIX ssn: <http://www.w3.org/ns/ssn/>

      SELECT $this ?ratio
      WHERE {
        ssp:strainerPlateAreaRatio ssn:isPropertyOf $this .
        ssp:strainerPlateAreaRatio ssp:hasVariableValue ?ratio .
        FILTER(?ratio < 4.0)
      }
    """
  ] .
```

Static calculation check

Input requirement.

A stored engineering coefficient shall match a regulation-derived calculated value. In this example, the expected coefficient is calculated from the height of the load area h and frame spacing s using $expected = 1 - 0.2 \cdot h/s$. The stored value in the ship graph must match the calculated value within a small numerical tolerance.

Output pattern.

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix ssp: <https://w3id.org/mtl-requirements/ssp#> .
@prefix ice: <http://example.com/iceregulations#> .
@prefix ssn: <http://www.w3.org/ns/ssn/> .

ice:LoadDistributionFactorShape a sh:NodeShape ;
```

```

sh:targetNode ssp:Hull001 ;
sh:message "Computed load-distribution factor validation failed." ;

sh:sparql [
  sh:message "Stored factor ({?actualFactor}) does not match the
calculated value ({?calculatedFactor}) based on load height and frame
spacing." ;
  sh:select """
    PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
    PREFIX ssn: <http://www.w3.org/ns/ssn/>

    SELECT $this ?actualFactor ?calculatedFactor
    WHERE {
      ssp:loadDistribution ssn:isPropertyOf $this .
      ssp:loadDistribution ssp:hasVariableValue ?actualFactor .

      ssp:loadArea ssn:isPropertyOf $this .
      ssp:loadArea ssp:hasVariableValue ?h .

      ssp:frameSpacing ssn:isPropertyOf $this .
      ssp:frameSpacing ssp:hasVariableValue ?s .

      BIND((1 - (0.2 * (?h / ?s))) AS ?calculatedFactor)
      FILTER(ABS(?actualFactor - ?calculatedFactor) > 0.01)
    }
  """
] .

```

Complex readiness check

Input requirement.

The engine output requirement shall be calculated by formula 3.1. The formula requires R_{CH} , the ice resistance in Newton of the ship in a channel with brash ice and a consolidated surface layer.

Output pattern.

```

ice:IceResistanceReadinessShape a sh:NodeShape ;
  sh:targetNode ssp:Hull001 ;
  sh:message "Simulation Readiness Check: Ice resistance output is missing
." ;

sh:sparql [
  sh:message "The hull is missing the ssp:IceResistance variable
required for the ice-resistance calculation." ;
  sh:select """
    PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
    PREFIX ssn: <http://www.w3.org/ns/ssn/>

    SELECT $this
    WHERE {
      FILTER NOT EXISTS {

```

```

        ssp:IceResistance ssn:isPropertyOf $this .
    }
}
""
] .

```

Dynamic readiness check

Input requirement.

Time-domain calculations shall be calculated for the MCR condition, MCR bollard conditions, and blade-order resonant rotational speeds so that the resonant vibration responses can be obtained.

Output pattern.

```

ice:TorsionalVibrationReadinessShape a sh:NodeShape ;
  sh:targetNode ssp:MainPropeller ;
  sh:message "Simulation Readiness Check: Peak propeller torque from
torsional vibration analysis is missing." ;

  sh:sparql [
    sh:message "Propeller is missing ssp:PeakPropellerTorque required
for time-domain torsional vibration analysis." ;
    sh:select ""
      PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
      PREFIX ssn: <http://www.w3.org/ns/ssn/>

      SELECT $this
      WHERE {
        FILTER NOT EXISTS {
          ssp:PeakPropellerTorque ssn:isPropertyOf $this .
        }
      }
    ""
  ] .

```

B Natural-language regulatory inputs used in the benchmark

This appendix lists the natural-language regulatory inputs used in the benchmark dataset. Only the input number and regulation text are shown to keep the appendix readable; implementation-specific JSON fields, file names, categories, and internal identifiers are omitted.

Table B1: Natural-language regulatory inputs used in the benchmark dataset.

Input	Regulation text
Input 1	Values of K_e for conventional propulsion systems shall match Table 3-1 based on propeller count and propulsion type.
Input 2	$C_{\mu} = 0.15 \cos \phi^2 + \sin \psi \sin \alpha$, and C_{μ} shall have a value equal to or larger than 0.45.
Input 3	If the value of the term $(LT/B^2)^3$ is less than 5, the value 5 shall be used and if the value of the term is more than 20, the value 20 shall be used.
Input 4	Under section 3 of the Act on the Ice Classes of Ships and Icebreaker Assistance (1121/2005), ships are assigned to ice classes IA Super, IA, IB, IC, II, or III.
Input 5	$HM = 1.0$ m for ice classes IA and IA Super, $= 0.8$ m for ice class IB, $= 0.6$ m for ice class IC.
Input 6	The design ice pressure is determined by the formula $p = cd * cp * ca * p_0$ [MPa], where p_0 is the nominal ice pressure and the value 5.6 MPa shall be used.
Input 7	cd is a factor which takes account of the influence of the size and engine output of the ship. The value of this factor is a maximum of $cd = 1$. It is calculated using the formula $cd = (a * k + b) / 1000$.
Input 8	cp is a factor that reflects the magnitude of the load expected in the hull area in question relative to the bow area. The value of cp for the bow region is given in Table 4-3.
Input 9	cp is a factor that reflects the magnitude of the load expected in the hull area in question relative to the bow area. The value of cp for the midbody region is given in Table 4-3.
Input 10	cp is a factor that reflects the magnitude of the load expected in the hull area in question relative to the bow area. The value of cp for the stern region is given in Table 4-3.
Input 11	The web thickness of the frames shall be at least the maximum of the following: $h_w * \sqrt{(\sigma_y) / C}$; half of the net thickness of the shell plating, $t - t_c$; and 9 mm.
Input 12	The stem and the part of a blunt bow defined above shall be supported by floors or brackets spaced no more than 0.6 m apart and with a thickness of at least half the plate thickness.

Continued on next page

Input	Regulation text
Input 13	In order to retain ice class IB or IC a ship, to which ice class regulations 1985 apply, shall comply with the required minimum engine output as defined in section 3.2.1 of the ice class regulations 1985. The engine output shall be no less than that determined by the formula below and in no case less than 740 kW for ice classes IB and IC.
Input 14	The capacity of the air receivers must be sufficient to provide no less than 12 consecutive starts of the propulsion engine without reloading, if it has to be reversed for moving astern.
Input 15	The capacity of the air receivers must be sufficient to provide no less than 6 consecutive starts of the propulsion engine without reloading, if the propulsion engine does not have to be reversed for moving astern.
Input 16	The capacity of the air compressors must be sufficient for charging the air receivers from atmospheric to full pressure in one (1) hour, except for a ship with ice class IA Super, if its propulsion engine has to be reversed for going astern, in which case the compressor must be able to charge the receivers in half an hour.
Input 17	The maximum service speed of the ship to be used in these calculations shall not be given a value lower than: IA Super 20 knots, IA 18 knots, IB 16 knots, IC 14 knots.
Input 18	If the actual maximum service speed of the ship is higher, that speed shall be used.
Input 19	For ice classes IA and IA Super, the rudder (the rudder stock and the upper part of the rudder) shall be protected from direct contact with intact ice by an ice knife that extends below the LIWL, if practicable (or equivalent means).
Input 20	For ice classes IA and IA Super, due regard shall be paid to the large loads that arise when the rudder is forced out of the midship position when sailing astern in ice or into ice ridges. Suitable arrangements such as rudder stoppers shall be installed to absorb such loads.
Input 21	Relief valves for the hydraulic pressure in rudder turning mechanism(s) shall be installed.
Input 22	For this purpose, at least one cooling water inlet chest shall be arranged as follows: the sea inlet shall be situated near the centreline of the ship and well aft, if possible.
Input 23	The ship shall declare at least one propulsion engine component so that the required engine output can be represented and verified against the regulation.
Input 24	The propulsion engine shall declare an engine power variable so that the required engine output can be represented and verified against the regulation.
Input 25	The section modulus of a stringer situated within the ice belt shall be calculated using the formula $Z = (f_6 \cdot f_7 \cdot p \cdot h \cdot l^2) / (m \cdot \sigma_y) \cdot 10^6$ [cm ³], and the effective shear area shall be $A = (\sqrt{3} \cdot f_6 \cdot f_7 \cdot f_8 \cdot p \cdot h \cdot l) / (2 \cdot \sigma_y) \cdot 10^4$ [cm ²].

Continued on next page

Input	Regulation text
Input 26	The forward draught shall be at least $(2 + 0.00025 \Delta) h_i$ [m], but need not exceed $4 h_i$.
Input 27	The height of the ice load area shall be as given in Table 4-1 for the relevant ice class.
Input 28	The maximum service speed of the ship to be used in these calculations shall not be given a value lower than IA Super 20 knots, IA 18 knots, IB 16 knots, and IC 14 knots.
Input 29	The highest point of the propeller shall be submerged and if possible at a depth of at least h_i below the water surface in all loading conditions.
Input 30	Propeller-hull clearance shall satisfy the minimum class-dependent clearance value used for ice operation calculations.
Input 31	Factor k shall equal $\sqrt{\Delta \cdot P} / 1000$, where Δ is the displacement of the ship and P is the engine output.
Input 32	Factor c_a shall equal $\sqrt{l_0 / l_a}$, where $l_0 = 0.6$ m, but c_a shall not be taken less than 0.35 and need not exceed 1.0.
Input 33	If the product $p \cdot h$ is less than 0.15, value 0.15 [MN/m] shall be used in stringer calculations.
Input 34	The vertical extension of the ice belt shall be as given in Table 4-5.
Input 35	The vertical extension of the ice belt shall be as given in Table 4-5.
Input 36	The vertical extension of the ice strengthening of framing shall be at least as given in Table 4-6.
Input 37	The vertical extension of the ice strengthening of framing shall be at least as given in Table 4-6.
Input 38	The maximum design ice block entering the propeller is a rectangular ice block with the dimensions $H_{ice} \cdot 2H_{ice} \cdot 3H_{ice}$. The thickness of the ice block (H_{ice}) is given in Table 6-3.
Input 39	Blade stress acceptability criterion requires $\sigma_{ref2} / \sigma_{st} \geq 1.3$.
Input 40	Fatigue acceptability criterion requires $\sigma_{fl} / \sigma_{fat} \geq 1.5$ at all blade locations.
Input 41	R_{CH} is the ice resistance in Newton of the ship in a channel with brash ice and a consolidated surface layer.
Input 42	The hull geometry parameters L , L_{BOW} , L_{PAR} , B , A_{wf} , α , ϕ_1 and ϕ_2 shall be defined for the engine output and ice resistance calculations.
Input 43	$P_{min} = K_e ((R_{CH} / 1000)^{(3/2)}) / D_p$ [kW], where K_e shall be given a value according to Table 3-1.
Input 44	The engine output requirement shall be calculated for two draughts by formula 3.1. Draughts to be used are the maximum draught amidships referred to as UIWL and the minimum draught amidships referred to as LIWL, and the engine output shall be no less than the greater of these two outputs.
Input 45	The design requirement for all ice classes is a minimum speed of 5 knots in the specified brash ice channel conditions.
Input 46	The design ice pressure shall be determined by the formula $p = c_d \cdot c_p \cdot c_a \cdot p_0$.

Continued on next page

Input	Regulation text
Input 47	For transversely framed plating in the ice belt, the plate thickness shall be calculated according to formula 4.5.
Input 48	For longitudinally framed plating in the ice belt, the plate thickness shall be calculated according to formula 4.6.
Input 49	The section modulus and shear area of transverse frames shall be calculated according to formulae 4.7 and 4.8.
Input 50	The section modulus and shear area of transverse frames shall be calculated according to formulae 4.7 and 4.8.
Input 51	The section modulus of longitudinal frames shall be calculated according to formula 4.9.
Input 52	The effective shear area of longitudinal frames shall be calculated according to formula 4.10.
Input 53	Web frames shall be designed for the maximum ice load F calculated in section 4.6.1.
Input 54	The effective shear area of web frames shall be calculated according to formula 4.16.
Input 55	The section modulus of web frames shall be calculated according to formula 4.17.
Input 56	The plate thickness of a shaped plate stem and, in the case of a blunt bow, the relevant part of the shell shall be calculated according to formula 4.5.
Input 57	The propeller blade scantling shall be based on the maximum lifetime backward and forward blade forces resulting from propeller/ice interaction, including hydrodynamic loads on the blade.
Input 58	The propeller thrust design shall include maximum backward propeller ice thrust T _b and maximum forward propeller ice thrust T _f during the ship's service life.
Input 59	The propeller design shall include the design ice torque Q _{max} resulting from propeller/ice interaction.
Input 60	D _p is the diameter of the propeller.
Input 61	The engine output P is the maximum output the propulsion machinery can continuously deliver to the propeller(s). If the output is restricted by technical means or regulations, P shall be taken as the restricted output.
Input 62	The hull shall identify the region under consideration as bow, midbody or stern for ice load and structural design calculations.
Input 63	Direct calculations may be used provided the structural response is evaluated against the material acceptance criterion based on yield stress.
Input 64	The allowable shear stress for direct analysis shall be based on the material yield stress, using $\tau_y = \sigma_y / \sqrt{3}$.
Input 65	If the supported stringer is outside the ice belt, the load effect shall be evaluated using the outside-ice-belt reduction and the section modulus shall satisfy the applicable formula.
Input 66	If the supported stringer is outside the ice belt, the load effect shall be evaluated using the outside-ice-belt reduction and the effective shear area shall satisfy the applicable formula.

Continued on next page

Input	Regulation text
Input 67	For an open propeller, the maximum lifetime backward blade force F_b shall be determined according to the applicable open-propeller formulae.
Input 68	For an open propeller, the maximum lifetime forward blade force F_f shall be determined according to the applicable open-propeller formulae.
Input 69	For a ducted propeller, the maximum lifetime backward blade force F_b shall be determined according to the applicable ducted-propeller formulae.
Input 70	For a ducted propeller, the maximum lifetime forward blade force F_f shall be determined according to the applicable ducted-propeller formulae.
Input 71	The design shall include the maximum spindle torque of the blade Q_{smax} for the ship's service life.
Input 72	The design shall include the number of relevant ice load cycles for lifetime and fatigue calculations.
Input 73	The propeller design shall include the maximum backward propeller ice thrust T_b during the ship's service life.
Input 74	The propulsion system design shall include the maximum response thrust T_r along the shaft line for the forward load case.
Input 75	The propulsion system design shall include the maximum response thrust T_r along the shaft line for the backward load case.
Input 76	For a controllable pitch propeller, the design shall include the propeller pitch at 0.7R radius at MCR in bollard condition, $P_{0.7b}$.
Input 77	The design shall include the equivalent contact radius calculation using the design ice block impacting the thruster, H_{iced} .
Input 78	The reference strength σ_{ref2} shall be taken as $0.7 \sigma_u$ or $0.6 \sigma_{0.2} + 0.4 \sigma_u$, whichever is less.
Input 79	The fatigue design shall include the mean fatigue strength σ_{exp} of the blade material at 10^8 cycles to failure in sea water.
Input 80	The fatigue design shall include the equivalent fatigue ice load stress amplitude σ_{fat} for 10^8 stress cycles.
Input 81	The fatigue design shall include the characteristic fatigue strength σ_{fl} of the blade material.
Input 82	The hull shall have the upper ice waterline UIWL and the lower ice waterline LIWL defined.
Input 83	If there is no relevant first blade order torsional resonance in the operational speed range or in the range 20% above and 20% below the maximum operating speed (bollard condition), the maximum design torque may be estimated using the non-resonant shaft line formula with equivalent inertia terms I_e and I_t .
Input 84	Time domain calculations shall be calculated for the MCR condition, MCR bollard conditions and for blade order resonant rotational speeds so that the resonant vibration responses can be obtained.
Input 85	If there is first blade order torsional resonance in the operational speed range or in the range 20% above and 20% below the maximum operating speed (bollard condition), the design torque Q_{peak} of the shaft component shall be determined by means of torsional vibration analysis, using frequency domain calculation as one of the two alternative methods.

Continued on next page

Input	Regulation text
Input 86	Q_max is the maximum torque on a propeller during the service life of the ship resulting from ice/propeller interaction for open propellers.
Input 87	Q_max is the maximum torque on a propeller during the service life of the ship resulting from ice/propeller interaction for ducted propellers.
Input 88	The ultimate load resulting from blade failure as a result of plastic bending around the blade root shall be calculated using formula 6.35, or alternatively by means of an appropriate stress analysis, reflecting the non-linear plastic material behaviour of the actual blade.
Input 89	The maximum spindle torque due to a blade failure load acting at 0.8R shall be determined.
Input 90	If the propeller centreline depth minus the design ice block thickness is insufficient for IB or IC, the propulsion arrangement shall satisfy the more severe IA-level submersion requirement.

C Generated SHACL outputs for selected benchmark cases

This appendix shows representative generated SHACL outputs from benchmark cases that were manually judged correct. The examples are included to illustrate the type of executable rule packages produced by the pipeline. The natural-language regulatory inputs for these cases are listed in Appendix B; therefore, this appendix focuses on the generated SHACL artifacts rather than repeating the full input JSON or pipeline metadata.

The examples cover a direct static value check, a static calculation check, a complex readiness check, and a dynamic readiness check. They should be interpreted as representative accepted outputs rather than as complete evidence of automatic regulatory correctness.

Case 06: Static nominal ice pressure

Listing 2: Generated SHACL for Case 06 from the full NLTL pipeline in Run 1.

```

1 @prefix ice: <https://w3id.org/mtl-requirements/ice#> .
2 @prefix ssp: <https://w3id.org/mtl-requirements/ssp#> .
3 @prefix ssn: <http://www.w3.org/ns/ssn/> .
4 @prefix sh: <http://www.w3.org/ns/shacl#> .
5 @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
6
7 ice:NominalIcePressureShape a sh:NodeShape ;
8   sh:targetNode ssp:iceShip ;
9   sh:message "Nominal ice pressure p0 shall be 5.6 MPa according to the design ice pressure
10     formula  $p = cd * cp * ca * p0$ ." ;
11
12   sh:sparql [
13     sh:message "Nominal ice pressure p0 is {p0} MPa, but it shall be 5.6 MPa." ;
14     sh:select """
15       PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
16       PREFIX ssn: <http://www.w3.org/ns/ssn/>
17
18       SELECT $this ?p0
19       WHERE {
20         ssp:nominalIcePressure ssn:isPropertyOf $this .
21         ssp:nominalIcePressure ssp:hasVariableValue ?p0 .
22         FILTER(?p0 != 5.6)
23       }
24     """
25   ] .

```

Case 39: Static calculation blade stress acceptability

Listing 3: Generated SHACL for Case 39 from the full NLTL pipeline in Run 1.

```

1 @prefix sh: <http://www.w3.org/ns/shacl#> .
2 @prefix ssp: <https://w3id.org/mtl-requirements/ssp#> .
3 @prefix ssn: <http://www.w3.org/ns/ssn/> .
4 @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
5
6 ssp:BladeStressAcceptabilityShape a sh:NodeShape ;
7   sh:targetNode ssp:MainPropeller ;

```

```

8      sh:message "Blade stress acceptability criterion requires sigma_ref2 / sigma_st >= 1.3." ;
9
10     sh:sparql [
11         sh:message "Blade stress acceptability failed: sigma_ref2 / sigma_st = {?ratio} (
            sigma_ref2 = {?sigma_ref2}, sigma_st = {?sigma_st}) is below the required minimum 1.3, or
            sigma_st is zero/undefined." ;
12         sh:select """
13             PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
14             PREFIX ssn: <http://www.w3.org/ns/ssn/>
15             PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
16
17             SELECT $this ?sigma_ref2 ?sigma_st ?ratio
18             WHERE {
19                 ssp:referenceStrength2 ssn:isPropertyOf $this ;
20                     ssp:hasVariableValue ?sigma_ref2 .
21
22                 ssp:bladeStress ssn:isPropertyOf $this ;
23                     ssp:hasVariableValue ?sigma_st .
24
25                 BIND(xsd:double(?sigma_ref2) AS ?sigma_ref2Num)
26                 BIND(xsd:double(?sigma_st) AS ?sigma_stNum)
27
28                 FILTER(?sigma_stNum != 0)
29
30                 BIND((?sigma_ref2Num / ?sigma_stNum) AS ?ratio)
31
32                 FILTER(?ratio < 1.3)
33             }
34         """
35     ] .

```

Case 42: Complex hull geometry readiness

Listing 4: Generated SHACL for Case 42 from the full NLTL pipeline in Run 1.

```

1  @prefix ssp: <https://w3id.org/mtl-requirements/ssp#> .
2  @prefix ssn: <http://www.w3.org/ns/ssn/> .
3  @prefix sh: <http://www.w3.org/ns/shacl#> .
4  @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
5  @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
6  @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
7
8  ssp:HullGeometryReadinessShape a sh:NodeShape ;
9      sh:targetNode ssp:iceShip ;
10     sh:message "Simulation Readiness Check: Hull geometry parameters for engine output and ice
        resistance calculations are not fully defined." ;
11
12     sh:sparql [
13         sh:message "Hull geometry readiness failed: one or more required parameters (L, L_BOW,
            L_PAR, B, A_wf, alpha, phi_1, phi_2) are missing. Expected variables: ssp:shipLength, ssp:
            bowLength, ssp:parallelMidshipLength, ssp:maxBreadth, ssp:bowWaterlineArea, ssp:
            waterlineAngle, ssp:bowRake, ssp:stemRake, each linked via ssn:isPropertyOf ssp:iceShip." ;
14         sh:select """
15             PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
16             PREFIX ssn: <http://www.w3.org/ns/ssn/>
17
18             SELECT $this
19             WHERE {
20                 FILTER NOT EXISTS {
21                     ssp:shipLength ssn:isPropertyOf $this .
22                     ssp:bowLength ssn:isPropertyOf $this .
23                     ssp:parallelMidshipLength ssn:isPropertyOf $this .
24                     ssp:maxBreadth ssn:isPropertyOf $this .
25                     ssp:bowWaterlineArea ssn:isPropertyOf $this .
26                     ssp:waterlineAngle ssn:isPropertyOf $this .

```

```

27         ssp:bowRake          ssn:isPropertyOf $this .
28         ssp:stemRake         ssn:isPropertyOf $this .
29     }
30 }
31 ""
32 ] .

```

Case 84: Dynamic resonant shaft time-domain readiness

Listing 5: Generated SHACL for Case 84 from the full NLTL pipeline in Run 1.

```

1  @prefix ice: <https://w3id.org/mtl-requirements/ice#> .
2  @prefix ssp: <https://w3id.org/mtl-requirements/ssp#> .
3  @prefix ssn: <http://www.w3.org/ns/ssn/> .
4  @prefix sh: <http://www.w3.org/ns/shacl#> .
5  @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
6  @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
7  @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
8
9  ice:DynamicRequirement_84_dynamic_resonant_shaft_time_domain a sh:NodeShape ;
10     sh:targetNode ssp:MainPropeller ;
11     sh:message "Simulation Readiness Check: Time-domain shaft vibration responses for MCR, MCR
12         bollard, and blade-order resonant speeds are required but missing." ;
13
14     sh:sparql [
15         sh:message "Time-domain shaft vibration response at MCR condition (ssp:
16             TimeDomainShaftResponse_MCR) is missing for the main propeller; regulation 84
17             _dynamic_resonant_shaft_time_domain requires this result." ;
18         sh:select ""
19             PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
20             PREFIX ssn: <http://www.w3.org/ns/ssn/>
21
22             SELECT $this
23             WHERE {
24                 FILTER NOT EXISTS {
25                     ?resp ssp:timeDomainCondition "MCR" ;
26                     ssn:isPropertyOf $this .
27                 }
28             }
29         ""
30     ] ;
31
32     sh:sparql [
33         sh:message "Time-domain shaft vibration response at MCR bollard condition (ssp:
34             TimeDomainShaftResponse_MCR_Bollard) is missing for the main propeller; regulation 84
35             _dynamic_resonant_shaft_time_domain requires this result." ;
36         sh:select ""
37             PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
38             PREFIX ssn: <http://www.w3.org/ns/ssn/>
39
40             SELECT $this
41             WHERE {
42                 FILTER NOT EXISTS {
43                     ?resp ssp:timeDomainCondition "MCR_Bollard" ;
44                     ssn:isPropertyOf $this .
45                 }
46             }
47         ""
48     ] ;
49
50     sh:sparql [
51         sh:message "Time-domain shaft vibration response at blade-order resonant rotational
52             speeds (ssp:TimeDomainShaftResponse_BladeOrderResonantSpeed) is missing for the main
53             propeller; regulation 84_dynamic_resonant_shaft_time_domain requires this result." ;
54         sh:select ""

```

```
48     PREFIX ssp: <https://w3id.org/mtl-requirements/ssp#>
49     PREFIX ssn: <http://www.w3.org/ns/ssn/>
50
51     SELECT $this
52     WHERE {
53         FILTER NOT EXISTS {
54             ?resp ssp:timeDomainCondition "BladeOrderResonantSpeed" ;
55                 ssn:isPropertyOf $this .
56         }
57     }
58     ""
59 ] .
```

D Prompt templates used in the experiments

This appendix lists the prompt templates used in the evaluated configurations. The prompts are shown as templates because case-specific fields were inserted at runtime, including the target regulation JSON, few-shot examples, ship-graph context when available, syntax and execution results for the validator, and repair feedback when applicable. The full NLTL pipeline used the generator prompt in Listing 6 and the validator prompt in Listing 7. The two ablation configurations did not use a validator prompt; instead, they used separate one-shot generator prompts, shown in Listings 8 and 9.

Full NLTL pipeline prompts

Generator prompt with ship-graph context

Listing 6: Generator prompt template used by the full NLTL pipeline.

```

1 You are a SHACL generator for maritime regulation compliance.
2
3 You will receive:
4 - one target regulation JSON
5 - the ship graph path and ship graph Turtle
6 - relevant few-shot examples from the same category
7 - optional repair feedback from a previous failed iteration
8
9 The ship graph is usually master_ship_1.ttl. Treat this as the canonical master ship design
  graph for this experiment. It is provided as graph context for vocabulary grounding, target
  selection, variable names, namespaces, and RDF value-access patterns. It is not merely a
  pass-test file. Do not infer new vocabulary from the regulation text when the master ship
  graph already shows the implemented RDF names and paths.
10
11 Your task:
12 Generate SHACL Turtle that faithfully represents the target regulation and is executable against
  the provided ship design graph.
13
14 Important requirements:
15 1. Output only SHACL Turtle between the tags <BEGIN_SHACL> and <END_SHACL>.
16 2. Do not output markdown fences.
17 3. Do not output explanations.
18 4. The SHACL must be syntactically valid Turtle.
19 5. Prefer SHACL-SPARQL when the regulation involves numeric thresholds, comparisons, arithmetic,
  formulas, or conditional logic.
20 6. Reuse the style and modeling patterns from the few-shot example when appropriate, but the
  provided master ship graph is the source of truth for targets, variable names, paths,
  namespaces, and value structure.
21 7. Be faithful to the regulation text. Do not invent extra requirements.
22 8. If repair feedback is provided, fix the identified issue rather than rewriting the task
  arbitrarily.
23 9. The generated rule should be suitable for checking conformance of ship design graphs against
  the target regulation.
24 10. Include useful sh:message text that reflects the actual failing reason, especially threshold
  /value mismatches.
25 11. Use standard prefixes when needed, including at minimum sh:, rdf:, rdfs:, xsd:, ssp:, and
  ssn:. Declare every prefix used, including ice: if you use it.
26 12. If the exact domain vocabulary is uncertain, stay conservative and consistent with the
  master ship graph.
27
28 Ship graph alignment rules:
29 13. Do not invent class names, property names, or direct property paths when the master ship
  graph already shows the actual names.

```

```

30 14. Prefer targets that actually exist in the master ship graph, such as sh:targetNode ssp:
    iceShip, ssp:Hull001, ssp:Engine001, ssp:MainPropeller, or ssp:Rudder001, or sh:targetClass
    only when the graph contains nodes with that rdf:type.
31 15. Most engineering quantities in this graph are modeled as variable nodes, not direct
    properties. The usual pattern is:
32
33 ?var ssn:isPropertyOf ?component ;
34     ssp:hasVariableValue ?value .
35
36 Components may also list variables with ssp:hasVariable ?var.
37
38 16. For readiness requirements, check that the correct existing variable node is present for the
    correct component and has ssp:hasVariableValue. Example pattern:
39
40 FILTER NOT EXISTS {
41     ssp:equivalentFatigueStress ssn:isPropertyOf $this ;
42     ssp:hasVariableValue ?value .
43 }
44
45 17. Use exact case-sensitive names from the master ship graph, for example ssp:KeValue, ssp:
    cMuCoefficient, ssp:brashIceThickness, ssp:propellerDiameter, ssp:propellerIsOpenType, ssp:
    characteristicFatigueStrength, and ssp:equivalentFatigueStress.
46 18. Applicability must match the master ship graph. For example, open propeller is represented
    by ssp:propellerIsOpenType ssn:isPropertyOf ssp:MainPropeller ; ssp:hasVariableValue true,
    not by a separate ssp:OpenPropeller class.
47 19. If the regulation needs data that is not present in the master ship graph, still produce
    syntactically valid SHACL, but do not fake unrelated targets or invented variable names;
    check the closest regulation-specific variable only when it is present in the graph.
48
49 Runtime fields:
50 TARGET_REGULATION_JSON:
51 {target_regulation_json}
52
53 SHIP_GRAPH_PATH:
54 {ship_graph_path}
55
56 SHIP_GRAPH_TURTLE:
57 {ship_graph_turtle}
58
59 FEW_SHOT_EXAMPLES:
60 {few_shot_examples}
61
62 REPAIR_FEEDBACK:
63 {repair_feedback_if_available}
64
65 Return format:
66 <BEGIN_SHACL>
67 ...turtle here...
68 <END_SHACL>

```

Validator prompt

Listing 7: Validator prompt template used by the full NLTL pipeline.

```

1 You are a validator for generated SHACL constraints in a maritime compliance pipeline.
2
3 You will receive:
4 - one target regulation JSON
5 - the generated SHACL
6 - the syntax result
7 - one ship graph validation result
8 - the ship graph path
9 - the ship graph Turtle
10 - relevant few-shot examples
11

```

12 Your job:

13 Evaluate whether the generated SHACL is acceptable for the target regulation and whether the
observed ship result is correct for this ship.

14

15 You must judge:

16 1. semantic faithfulness to the regulation

17 2. whether syntax problems are real and important

18 3. what the expected outcome on this ship should be under the regulation:

19 - pass

20 - fail

21 - not_applicable

22 - unknown

23 4. what the actual SHACL outcome was:

24 - pass

25 - fail

26 - execution_error

27 - unknown

28 5. whether the observed ship behavior is correct

29 6. whether the observed result happened for the right reason implied by the regulation

30 7. whether applicability was handled correctly

31 8. whether the result should be accepted, retried, or rejected

32

33 Important interpretation rule:

34 A generated SHACL can be correct even when the ship fails validation. A generated SHACL can also
be wrong even when the ship passes validation. Judge correctness by comparing the
regulation meaning against the ship facts and the observed SHACL behavior.

35

36 If the ship graph provides a precomputed regulatory value directly, such as `ssp:cMuCoefficient`,
`ssp:KeValue`, or another named coefficient, the SHACL may validate that value directly
against the regulatory threshold or table. Do not require the SHACL to recompute the value
from lower-level formula inputs unless the target regulation JSON explicitly asks for
formula reconstruction and the required input variables exist in the ship graph.

37

38 You must explicitly reason over these four cases:

39 1. rule correct + ship passes -> accept

40 2. rule correct + ship fails -> accept

41 3. rule wrong + ship passes by accident -> retry or reject

42 4. rule wrong + ship fails by accident -> retry or reject

43

44 Decision guidance:

45 - accept:

46 semantics are correct enough, syntax is valid, the SHACL is executable, and the observed ship
result matches the expected outcome for the right reason

47 - retry:

48 the rule is fixable, partially correct, has reporting/modeling issues, has unclear
applicability handling, or the expected/actual outcome mismatch looks repairable

49 - reject:

50 the semantics are fundamentally wrong, the rule validates the wrong thing entirely, or the
behavior clearly contradicts the regulation

51

52 Important rules:

53 1. Do not regenerate the SHACL from scratch.

54 2. Do not output markdown.

55 3. Output valid JSON only.

56 4. Be strict about semantic mismatch.

57 5. If the observed result appears to come from the wrong target, wrong variable, wrong threshold
, wrong enum set, wrong applicability condition, or wrong modeling assumption, do not
accept.

58 6. If the SHACL appears structurally valid but does not really encode the regulation, do not
accept.

59 7. If the ship result is `execution_error`, do not accept.

60 8. If `expected_outcome` is `unknown` because the ship facts are insufficient, usually choose retry
unless the semantic mismatch is fundamental.

61 9. If the regulation does not apply to this ship, judge whether the SHACL handled that correctly
rather than blindly rewarding pass/fail.

62

63 You must explain your judgment briefly but concretely. Use the ship graph, the regulation, and
the SHACL behavior. State which facts from the ship you relied on. The justification must


```

        explain why the expected outcome should be pass, fail, or not_applicable. Do not give
        generic explanations. Cite concrete ship facts such as variable names, values, targets, and
        applicability conditions when available.
64
65 Runtime fields:
66 TARGET_REGULATION_JSON:
67 {target_regulation_json}
68
69 GENERATED_SHACL:
70 {generated_shacl}
71
72 SYNTAX_RESULT:
73 {syntax_result}
74
75 SHIP_VALIDATION_RESULT:
76 {ship_validation_result}
77
78 SHIP_GRAPH_PATH:
79 {ship_graph_path}
80
81 SHIP_GRAPH_TURTLE:
82 {ship_graph_turtle}
83
84 FEW_SHOT_EXAMPLES:
85 {few_shot_examples}
86
87 Required JSON format:
88 {
89   "decision": "accept|retry|reject",
90   "confidence": 0.0,
91   "semantic_match": true,
92   "syntax_valid": true,
93   "expected_outcome": "pass|fail|not_applicable|unknown",
94   "actual_outcome": "pass|fail|execution_error|unknown",
95   "ship_behavior_correct": true,
96   "reason_alignment": true,
97   "applicability_handled_correctly": true,
98   "facts_used": [],
99   "regulation_interpretation": "",
100  "applicability_explanation": "",
101  "justification": "",
102  "issues": [],
103  "suggested_fix": ""
104 }

```

Generator prompt for NLTL without validator

Listing 8: Generator prompt template used by NLTL without validator.

```

1 You are a SHACL generator for maritime regulation compliance.
2
3 You will receive:
4
5 * one target regulation JSON
6 * the ship graph path and ship graph Turtle
7 * relevant few-shot examples from the same category
8
9 This is a one-shot ablation configuration. There is no validator loop, no retry step, and no
  repair feedback. Therefore, generate the best possible SHACL artifact in the first attempt.
10
11 The ship graph is usually master_ship_1.ttl. Treat this as the canonical master ship design
   graph for this experiment. It is provided as graph context for vocabulary grounding, target
   selection, variable names, namespaces, and RDF value-access patterns. It is not merely a
   pass-test file. Do not infer new vocabulary from the regulation text when the master ship
   graph already shows the implemented RDF names and paths.

```

Your task:

Generate SHACL Turtle that faithfully represents the target regulation and is executable against the provided ship design graph.

Important requirements:

1. Output only SHACL Turtle between the tags <BEGIN_SHACL> and <END_SHACL>.
2. Do not output markdown fences.
3. Do not output explanations.
4. The SHACL must be syntactically valid Turtle.
5. Prefer SHACL-SPARQL when the regulation involves numeric thresholds, comparisons, arithmetic, formulas, or conditional logic.
6. Reuse the style and modeling patterns from the few-shot example when appropriate, but the provided master ship graph is the source of truth for targets, variable names, paths, namespaces, and value structure.
7. Be faithful to the regulation text. Do not invent extra requirements.
8. The generated rule should be suitable for checking conformance of ship design graphs against the target regulation.
9. Include useful sh:message text that reflects the actual failing reason, especially threshold/value mismatches.
10. Use standard prefixes when needed, including at minimum sh:, rdf:, rdfs:, xsd:, ssp:, and ssn:. Declare every prefix used, including ice: if you use it.
11. If the exact domain vocabulary is uncertain, stay conservative and consistent with the master ship graph.

Ship graph alignment rules:

12. Do not invent class names, property names, or direct property paths when the master ship graph already shows the actual names.
13. Prefer targets that actually exist in the master ship graph, such as sh:targetNode ssp:iceShip, ssp:Hull001, ssp:Engine001, ssp:MainPropeller, or ssp:Rudder001, or sh:targetClass only when the graph contains nodes with that rdf:type.
14. Most engineering quantities in this graph are modeled as variable nodes, not direct properties. The usual pattern is:

```
?var ssn:isPropertyOf ?component ;
ssp:hasVariableValue ?value .
```

Components may also list variables with ssp:hasVariable ?var.

15. For readiness requirements, check that the correct existing variable node is present for the correct component and has ssp:hasVariableValue. Example pattern:

```
FILTER NOT EXISTS {
  ssp:equivalentFatigueStress ssn:isPropertyOf $this ;
  ssp:hasVariableValue ?value .
}
```

16. Use exact case-sensitive names from the master ship graph, for example ssp:KeValue, ssp:cMuCoefficient, ssp:brashIceThickness, ssp:propellerDiameter, ssp:propellerIsOpenType, ssp:characteristicFatigueStrength, and ssp:equivalentFatigueStress.

17. Applicability must match the master ship graph. For example, open propeller is represented by ssp:propellerIsOpenType ssn:isPropertyOf ssp:MainPropeller ; ssp:hasVariableValue true, not by a separate ssp:OpenPropeller class.

18. If the regulation needs data that is not present in the master ship graph, still produce syntactically valid SHACL, but do not fake unrelated targets or invented variable names; check the closest regulation-specific variable only when it is present in the graph.

Runtime fields:

```
TARGET_REGULATION_JSON:
{target_regulation_json}
```

```
SHIP_GRAPH_PATH:
{ship_graph_path}
```

```
SHIP_GRAPH_TURTLE:
{ship_graph_turtle}
```

```

61 FEW_SHOT_EXAMPLES:
62 {few_shot_examples}
63
64 Return format:
65 <BEGIN_SHACL>
66 ...turtle here...
67 <END_SHACL>

```

Generator prompt for NLTL without validator and without ship context

Listing 9: Generator prompt template used by NLTL without validator and without ship context.

```

1  You are a SHACL generator for maritime regulation compliance.
2
3  You will receive:
4
5  * one target regulation JSON
6  * relevant few-shot examples from the same category
7
8  This is a one-shot ablation configuration. There is no validator loop, no retry step, no repair
9  feedback, and no ship graph Turtle. Therefore, generate the best possible SHACL artifact in
10 the first attempt using only the target regulation and few-shot examples.
11
12 No ship graph Turtle is provided in this configuration. Therefore, do not rely on case-specific
13 graph inspection. Generate a conservative SHACL Turtle artifact using only the target
14 regulation, the requirement category, and the few-shot examples. Use the same general
15 namespace and modeling conventions as the few-shot examples when appropriate.
16
17 Your task:
18 Generate SHACL Turtle that faithfully represents the target regulation as far as possible
19 without target-graph context.
20
21 Important requirements:
22
23 1. Output only SHACL Turtle between the tags <BEGIN_SHACL> and <END_SHACL>.
24 2. Do not output markdown fences.
25 3. Do not output explanations.
26 4. The SHACL must be syntactically valid Turtle.
27 5. Prefer SHACL-SPARQL when the regulation involves numeric thresholds, comparisons, arithmetic,
28 formulas, or conditional logic.
29 6. Reuse the style and modeling patterns from the few-shot examples when appropriate.
30 7. Be faithful to the regulation text. Do not invent extra requirements.
31 8. The generated rule should be suitable as a draft SHACL rule for later checking against a ship
32 design graph.
33 9. Include useful sh:message text that reflects the actual failing reason, especially threshold/
34 value mismatches.
35 10. Use standard prefixes when needed, including at minimum sh:, rdf:, rdfs:, xsd:, ssp:, and
36 ssn:. Declare every prefix used, including ice: if you use it.
37 11. If the exact domain vocabulary is uncertain, stay conservative and use the closest
38 vocabulary pattern shown in the few-shot examples.
39
40 No-context generation rules:
41 12. Do not claim that a graph-specific variable, component, or RDF path is known unless it
42 appears in the few-shot examples or target regulation JSON.
43 13. Prefer simple and transparent SHACL-SPARQL patterns over complicated invented graph
44 structures.
45 14. For readiness-style requirements, generate a rule that checks for the presence of the
46 required analysis variable, output, or evidence slot using the general variable-node
47 pattern from the examples.
48 15. For direct value or threshold requirements, generate a rule that compares the relevant value
49 against the regulatory threshold using a clearly named variable.

```

```
34 16. For formula-heavy requirements where all inputs are not available, generate a conservative
    stored-value or readiness-style draft rather than inventing a full unsupported formula
    model.
35 17. Do not invent applicability classes such as ssp:OpenPropeller unless the prompt examples
    support that modeling pattern. Prefer explicit value-based applicability when possible.
36 18. If the regulation cannot be fully represented from the available information, still produce
    syntactically valid SHACL but avoid adding unsupported assumptions.
37
38 Runtime fields:
39 TARGET_REGULATION_JSON:
40 {target_regulation_json}
41
42 FEW_SHOT_EXAMPLES:
43 {few_shot_examples}
44
45 Return format:
46 <BEGIN_SHACL>
47 ...turtle here...
48 <END_SHACL>
```